Marine Race Arena: A Reproducible Benchmark for Onboard-Only Autonomous Underwater Gate Racing, Multi-Vehicle Evaluation and Learned Control

Andrea Bedei[a], Lorenzo Bacchiani[a], Giovanni Pau[a], Roberto Girau[a]

[a]*Department of Computer Science and Engineering, University of Bologna, Italy*

Abstract

Autonomous racing has become a standard way to compare perception, planning and control on ground and aerial platforms, but underwater robotics still lacks an equivalent evaluation protocol. Marine simulators supply vehicles, hydrodynamics and sensors; none of them defines the surrounding contract that makes results comparable, namely an ordered gate course, a fixed observation boundary, an impartial scorer and machine-readable outcomes. We present Marine Race Arena, a configurable benchmark and evaluation framework for onboard-only autonomous underwater gate racing, implemented over a HoloOcean backend with a BlueROV2-class vehicle. The benchmark separates autonomy from evaluation by construction: a controller reads only a forward camera, depth, inertial and Doppler measurements, physically received acoustic beacon packets and optional teammate messages, while an independent referee uses privileged simulator state to validate ordered gate crossings, charge penalties, terminate runs and compute scores that are never fed back into control. On top of this contract we contribute an onboard gate-perception front end that associates camera detections with the expected acoustic beacon, two interpretable rule-based reference controllers, a cooperative fleet layer with a distributed leader–follower policy over an acoustic-inspired channel, and a learning-integration path that exposes the same observation and action contract to reinforcement-learning agents. We report 78 real-HoloOcean runs across three heterogeneous circuits, two graded current profiles, homogeneous fleets and three-vehicle coordination; a current-free demonstration in which the reference controller completes all three circuits in 9 of 9 runs with no collision, out-of-bounds or wrong-direction event; and a 474-episode matched comparison of six controllers under identical seeds and geometries. Three findings recur. Success on a short clean circuit does not predict robustness: the same controllers drop to 2–3 of 5 completions under a moderate current. Coordination matters more than speed in a convoy: a one-gate leader–follower margin removes all 127.3 mean gate and world collisions of the uncoordinated three-vehicle case. And learned policies are not simply worse than rules but differently shaped: they complete 90.6–100% of short generated geometries and are significantly faster there than the rule baseline, yet reach only 20–40% on the full circuits, failing almost exclusively by losing the gate sequence rather than by colliding. Marine Race Arena makes such distinctions measurable, and the released configurations, manifests and hashes make every reported number reproducible.

Keywords: underwater robotics, autonomous racing, robotics benchmark, reproducible evaluation, onboard perception, multi-robot systems, reinforcement learning, HoloOcean

1. Introduction

Autonomous racing has become an effective research format for evaluating perception, planning, control and real-time decision-making within a single quantitative task under repeatable conditions [1]. In ground robotics, the F1TENTH platform provides standardized vehicles, circuits, metrics and baselines for continuous control and reinforcement learning [2], while at full scale the Abu Dhabi Autonomous Racing League pushes planning and control toward the limits of vehicle handling in head-to-head scenarios [3]. In aerial robotics, gate-based benchmarks and competitions have driven progress from modular perception and control pipelines [4] to learning-based policies capable of champion-level drone racing [5]. What these efforts share is not a vehicle or an algorithm but a contract: a well-defined task, fixed course geometry, explicit scoring rules and machine-readable outcomes. That contract, more than any individual result, is what has made autonomy methods in those domains comparable, reproducible and cumulatively improvable.

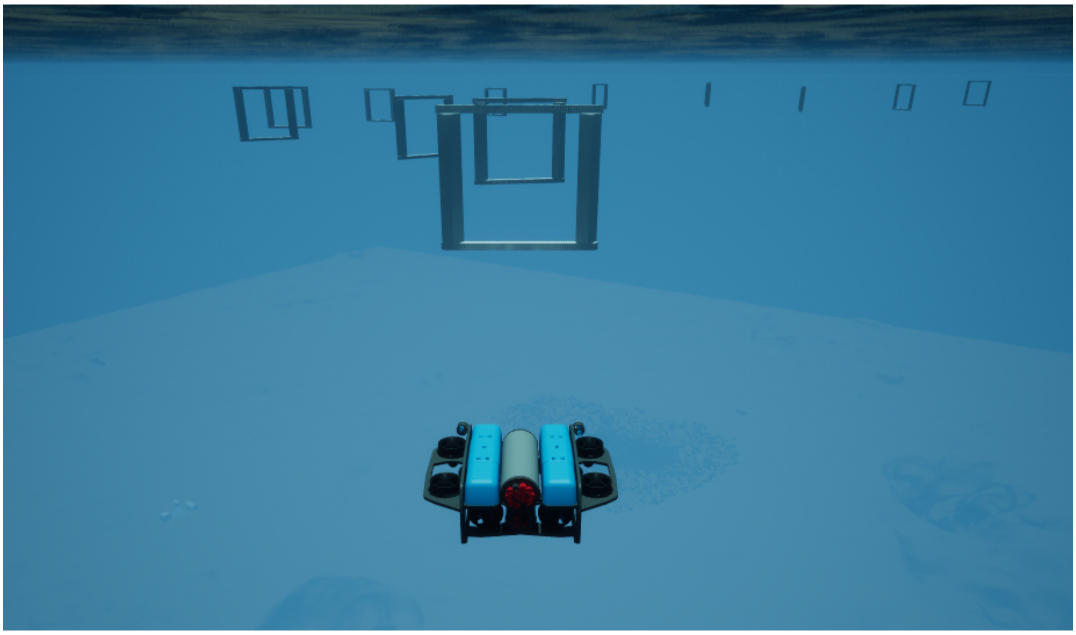

Figure 1: Underwater gate racing in Marine Race Arena. A BlueROV2-class vehicle approaches the ordered, beacon-marked gate line of an official circuit in HoloOcean. The controller sees this scene only through its forward camera, its depth, inertial and Doppler sensors and the acoustic packets it physically receives; the gate positions visible to the reader are never available to it.

Underwater robotics has no equivalent contract for speed-oriented autonomous navigation along a predefined gate course. Marine simulators have matured considerably: the UUV Simulator provides Gazebo-based intervention and multi-robot scenarios [6], Stonefish contributes marine-specific physics and a broad sensor suite [7], DAVE adds acoustic and optical sensing for autonomous underwater vehicles [8], HoloOcean renders underwater scenes in Unreal Engine with sonar, camera, inertial and Doppler sensing [9], and MarineGym delivers GPU-parallel hydrodynamics and Gym-style control tasks for scalable reinforcement-learning training [10]. These systems supply the physical and sensing substrate, and they supply it well. What none of them supplies is the evaluation layer that sits above a simulator: an ordered gate sequence, an enforced boundary on what the controller may observe, an impartial scorer that adjudicates progress, and a reproducible output protocol. Without that layer, two underwater controllers reported on the same simulator can still be incomparable, because nothing prevents one of them from consuming information that the other refused, or from being scored by logic it partly controls.

This omission is not a formality, because underwater racing is not aerial or ground racing transplanted into water. Underwater vehicles are slow, strongly damped and pushed persistently off line by currents; global localization is typically unavailable; inertial and Doppler measurements drift; optical perception is short-ranged and degraded by turbidity; acoustic cues are noisy, low-bandwidth and intermittent; and contact with a gate, an obstacle or a teammate is hard to attribute from onboard signals alone. Each of these properties changes what a controller can know, and therefore changes what an honest evaluation must withhold from it. A benchmark that leaks global pose or official course progress into the control loop measures a capability no deployed vehicle has. Conversely, a benchmark that withholds that information but scores progress with the same partial estimates the controller uses cannot detect when the controller is wrong. The two roles must be separated, and the separation must be enforced by the framework rather than left to the goodwill of the participant.

Marine Race Arena makes that separation the organizing principle of an underwater racing benchmark (Fig. 1). A declarative JSON configuration specifies the world, the ordered gates, the sensor profile, the acoustic beacon network, the disturbance field, the obstacles, the participants and the scoring rules. A controller receives an *official observation* containing only participant-local time, allow-listed onboard measurements, physically received beacon packets and optional teammate messages, and returns a bounded body-frame command. An *independent referee* reads privileged simulator state to test ordered gate crossings, detect violations, charge penalties, terminate runs and rank participants.

Referee-derived progress never returns to the controller; where the controller's own estimate of its progress disagrees with the referee, the disagreement is logged for analysis rather than corrected. The benchmark model, controller contract and referee are vehicle-independent, and a replaceable adapter isolates simulator-specific sensing and actuation; the reference implementation and every experiment reported here use HoloOcean [9] with a BlueROV2-class vehicle [11].

Building the evaluation layer first turns out to change what the experiments can say. Because the observation boundary is fixed, a rule-based controller, a hybrid controller and a neural policy can be run through the identical suite on identical seeds and geometries, and the resulting differences are attributable to the policy rather than to the information it was allowed. Because the referee is independent, the controller's own course estimate becomes a measurable quantity rather than an assumption: across the frozen benchmark matrix the controller-local progression matched the referee on 1467 of 1474 gate advancements, lagging it by a median of 0.858 s, with 11 false and 7 missed local advancements and no case in which a controller believed it had finished before the referee agreed. Because scoring is machine-readable, the same framework supports single-vehicle trials, cooperative fleets and learned controllers without changing how any of them is judged.

Three experimental observations recur across the study and motivate the structure of the evaluation. First, clean-track success is a weak predictor of robustness: both reference controllers complete every gate of the short Horseshoe Bay circuit in every seed, yet fall to 2 and 3 completions out of 5 under a moderate current, and neither completes a single run under the strong profile. Second, in a heterogeneous convoy the binding constraint is coordination rather than individual speed: an uncoordinated three-vehicle team records a mean of 127.3 gate and world collisions per run at an 8 s release gap, while a one-gate leader–follower margin using only legal onboard estimates and an acoustic-inspired channel removes those events entirely and is faster than a more conservative two-gate margin. Third, learned and rule-based controllers fail in qualitatively different ways: in a 474-episode matched comparison, learned policies complete 90.6–100% of short generated geometries and are significantly faster there than the rule baseline, but reach only 20–40% on the full official circuits, and almost every failure is a lost gate sequence rather than a collision. That asymmetry is invisible to a benchmark that reports only aggregate completion, and it is exactly the kind of distinction an evaluation contract is supposed to expose.

This paper extends a preliminary description of the benchmark with the perception front end, the referee formalization, the learning-integration path and the experimental programme summarized above. Its contributions are as follows.

1. **An underwater gate-racing benchmark defined as an evaluation contract rather than a simulator.** Marine Race Arena specifies a complete race scenario — world, ordered gates, sensor profile, acoustic beacons, currents, obstacles, participants, timing and scoring — in a single declarative configuration, and enforces the resulting contract at the framework boundary. Three official circuits of increasing length and vertical complexity ship with the release, and the benchmark model is independent of the simulator behind the adapter.
2. **A strict, framework-enforced information boundary between autonomy and evaluation.** We formalize the official observation, the command interface and the referee state, and show that the observation builder is structurally independent of the referee. We then *measure* the boundary rather than assert it, reporting the agreement between controller-local progression and independent referee progression over 1474 gate advancements.
3. **An onboard gate-perception and target-association front end.** A deterministic detector extracts candidate gate frames from the forward camera and reports normalized image geometry and a confidence; an association stage selects, among competing candidates, the one consistent with the bearing of the controller's own expected acoustic beacon, and rejects the rest. We audit the front end on all three circuits along the controller's own trajectory, and we report separately, and negatively, the limits of an exploratory pose-aware extension under strong gate obliquity.
4. **An independent automated referee with formal gate validation and scoring.** The referee tests ordered crossings by a signed-distance and aperture test on privileged state, arbitrates events in a fixed priority order, applies penalties and terminal conditions, ranks finished and unfinished participants by an explicit lexicographic key, and emits structured event logs and machine-readable summaries.
5. **A cooperative fleet layer with team-level scoring and a distributed coordination policy.** Multiple vehicles run one circuit as a single team with independent onboard autonomy and independent referee state. A leader–follower wrapper coordinates the convoy using only its own tracker state, a static predecessor assignment and

controller-authored messages over an acoustic-inspired channel with range-dependent latency and loss.

6. **A learning-integration path that reuses the same contract, with an honest account of where it currently stands.** We describe the observation encoding, reward, curriculum, training pipeline and pre-registered readiness gate used to train and select policies, and we evaluate learned, hybrid and rule-based controllers in one matched suite. We report both the regime in which learned policies win and the regime in which they fail, including a pre-registered gate that the frozen policy did not pass.
7. **A reproducible experimental programme over real HoloOcean runs.** The evaluation covers 78 runs of the frozen benchmark matrix, a nine-run current-free completion demonstration with a verified zero-current manifest, and a 474-episode matched controller comparison, each carrying commit identifiers, source-tree or model hashes, adapter and fallback status, and seed records.

The remainder of the paper is organized as follows. Section 2 positions Marine Race Arena against underwater simulation, racing benchmarks, underwater perception, multi-AUV cooperation and learning-based underwater control. Section 3 defines the benchmark instance, the participant interface and the runtime architecture. Section 4 describes the vehicle, sensing, acoustic and current models and the official observation contract. Section 5 presents the onboard gate-perception and association front end. Section 6 specifies the controller interface and the two rule-based reference controllers. Section 7 formalizes the referee and the information boundary. Section 8 describes fleet execution, team scoring and coordination. Section 9 presents the learning extension. Section 10 reports the experimental evaluation as five research questions. Sections 11, 12 and 13 discuss the findings, state the limitations and document reproducibility. Section 14 concludes.

2. Related Work

We review five lines of work that bound the contribution: underwater simulation, which supplies the physical substrate; autonomous racing and reproducible robotics benchmarks, which supply the evaluation format we adapt; underwater perception and sensor-based navigation, which determine what an onboard controller can actually observe; multi-AUV cooperation and acoustic communication, which constrain the fleet layer; and learning-based underwater control, which motivates the learning-integration path. Table 1 summarizes the positioning.

2.1. Underwater Robotics Simulation

Underwater simulators differ mainly in the physics engine they build on and in the breadth of the sensor suite they expose. The UUV Simulator packages Gazebo-based hydrodynamics for underwater intervention and multi-robot scenarios [6]. Stonefish replaces the general-purpose engine with marine-specific physics and exposes a wide range of underwater sensors behind a ROS interface [7]. DAVE builds on Gazebo to support acoustic and optical sensing for autonomous underwater vehicles [8]. HoloOcean uses Unreal Engine to render underwater scenes and exposes sonar, an RGB camera, an IMU, a DVL and a depth sensor for one or more agents [9]; it is the simulation backend used in this work, reached through a replaceable adapter.

These systems are complementary to Marine Race Arena rather than alternatives to it, because they answer a different question. They determine *what happens* when a vehicle acts in water: how it is dragged, what its sensors return, how a scene renders. They do not determine *how a result is judged*: which gates count, in which order, under what observation restriction, adjudicated by whom, and recorded in what form. A controller evaluated on any of them can be given world pose or exact obstacle geometry without violating anything, because no contract forbids it. Marine Race Arena adds precisely that missing layer, and adds it above the adapter boundary so that the same race model and referee could in principle be driven by any of these backends.

2.2. Autonomous Racing and Reproducible Robotics Benchmarks

Racing benchmarks are valuable because they expose end-to-end behaviour under repeatable, adversarial constraints, as catalogued in the survey of Betz et al. [1]. F1TENTH is the closest in spirit to this work: it standardizes a scaled autonomous car, circuits and an evaluation environment for continuous control and reinforcement learning [2]. It targets ground vehicles with planar LiDAR, however, and treats referee logic as a partial, in-framework concern rather than as a separated first-class contract. At full scale, the A2RL challenge frames racing as multi-agent interaction at the limits of vehicle handling [3], and the same league runs a drone-racing format [13]. In aerial robotics,

Table 1: Positioning of Marine Race Arena against representative underwater simulators and racing benchmarks. Criteria are properties of the published system description, not quality judgements; the table deliberately omits hydrodynamic fidelity, sensor variety and training throughput, on which several listed systems exceed Marine Race Arena.

System	Domain	Physics and sensing	Gate race	Indep. referee	Obs. contract	Multi-robot	Team score
UUV Simulator [6]	Underwater	Gazebo, hydrodynamics	no	no	no	✓	no
Stonefish [7]	Underwater	Custom marine physics, sensors, ROS	no	no	no	no	no
DAVE [8]	Underwater	Gazebo, acoustic and optical sensing	no	no	no	✓	no
MarineGym [10]	Underwater	Isaac Sim, GPU hydrodynamics, RL tasks	no	no	✓	partial	no
F1TENTH [2]	Ground	Scaled car, planar LiDAR, RL and control	✓	partial	✓	partial	no
AlphaPilot, Swift [4, 5]	Aerial	Quadrotor, vision-based gate racing	✓	partial	✓	no	no
CARLA [12]	Ground	Unreal Engine, urban sensing and tasks	partial	partial	✓	no	no
Marine Race Arena (this work)	Underwater	HoloOcean backend, BlueROV2, camera and acoustic sensing	✓	✓	✓	✓	✓

Gate race: an ordered gate course with timing is part of the defined task. *Indep. referee*: progress and violations are adjudicated by a component the participant cannot influence. *Obs. contract*: a documented observation interface; ✓ additionally denotes that ground-truth pose is excluded from control by the contract. *Team score*: a defined aggregate score over a group of vehicles. “Partial” marks a supported capability that is not a primary design contract of that system. All listed underwater simulators supply physics and sensing that Marine Race Arena itself does not provide and depends on a backend for.

AlphaPilot established ordered gate sequences as a demanding benchmark for fast perception and control [4], and Swift subsequently demonstrated champion-level drone racing with a policy trained by deep reinforcement learning against a simulated opponent [5]. Outside racing, CARLA illustrates how an Unreal-based ecosystem can standardize maps, sensors and tasks for driving research [12], and standardized environment interfaces such as OpenAI Gym [14] together with GPU-parallel robot-learning simulators such as Isaac Gym [15] show how a documented observation–action contract enables systematic comparison of learned controllers.

Marine Race Arena adopts the ordered-gate task and the documented observation contract from this line of work, and adapts them to a medium in which the assumptions behind the aerial and ground formulations do not hold. Drone-racing gates are acquired optically at long range in clear air and a vehicle can accelerate hard to recover a lost line; underwater, optical range is short, the vehicle is strongly damped, a persistent current biases every manoeuvre, and the most reliable long-range cue is a noisy, intermittent acoustic bearing rather than an image. The consequence for benchmark design is that gate acquisition and target association become first-class problems rather than preliminaries, and that a referee cannot rely on the controller’s own notion of progress.

2.3. Underwater Perception and Sensor-Based Navigation

Underwater autonomy is shaped by the fact that the sensors which are cheap in air are expensive or unavailable in water. Satellite positioning does not penetrate the surface, so global pose must be inferred acoustically or dead-reckoned from inertial and Doppler measurements, with drift that grows over a run. Optical sensing is limited in range and degraded by turbidity, scattering and wavelength-dependent attenuation, which is why simulators aimed at marine work model optical and acoustic sensing explicitly [7, 8, 9]. Acoustic ranging is the practical long-range cue, but it is noisy, low-rate, and subject to the propagation effects characterized in the underwater acoustic-channel literature [16].

Marine Race Arena does not propose a new underwater perception algorithm, and does not claim that its detector is a state-of-the-art gate detector. It makes a narrower methodological point that the benchmark setting requires: when several gates are simultaneously visible and the vehicle has only a noisy acoustic bearing to distinguish them, *which* detection the controller commits to is a decision the framework must not make on the controller’s behalf. The framework therefore delivers unlabelled camera pixels and unlabelled beacon packets, and the association between them is part of the participant's autonomy stack (Section 5). This is the concrete mechanism by which the observation contract stays honest under multi-gate visibility.

2.4. Multi-AUV Cooperation and Acoustic Communication

Fleet operation underwater is constrained by the physics of the acoustic channel and by uncertain relative localization. Underwater acoustic links suffer low bandwidth, long propagation delay, multipath and time-varying attenuation [16], so the assumption of continuous, high-bandwidth, low-latency inter-vehicle communication that underpins many terrestrial multi-robot methods does not transfer. Surveys of AUV formation control document how these communication and navigation constraints shape achievable coordination: Yang et al. analyse formation performance, control and communication capability jointly [17], and Yan et al. review formation-control methods and their dependence on relative-state estimation [18].

Consistent with this literature, Marine Race Arena assumes neither shared ground-truth state nor high-rate communication. Its fleet mode runs several vehicles as one cooperative team with staggered or simultaneous releases, independent per-vehicle onboard autonomy and independent per-vehicle referee state, referee-side inter-vehicle proximity detection that is never exposed to controllers, team-level scoring, an acoustic-inspired inter-vehicle channel with range-dependent latency and loss, and a distributed leader–follower controller built on top of that channel (Section 8). The channel is deliberately presented as transport for coordination experiments and not as a characterized channel model: its packet loss, range, latency and freshness parameters are not independently swept in this work.

2.5. Learning-Based Underwater Control and Simulation

Learning-based control for marine vehicles has been enabled by simulators that expose Gym-style interfaces and sufficient throughput. MarineGym is the most directly relevant recent system: it extends Isaac Sim with GPU-accelerated hydrodynamics, multiple UUV models, domain randomization and benchmark tasks for station keeping, trajectory tracking and docking, and reports large speedups that make reinforcement-learning training on underwater vehicles practical [10]. The broader pattern it belongs to — a documented observation–action interface plus parallel simulation — is the same one that made ground and aerial learned control comparable across groups [14, 15, 5].

MarineGym and Marine Race Arena address different halves of the same problem and compose naturally. MarineGym is a *training* platform: its contribution is throughput, hydrodynamic fidelity, randomization and a set of standardized underwater control tasks, and it is designed so that a policy can see many environment steps quickly. Marine Race Arena is an *evaluation* framework: its contribution is a racing protocol over ordered gates, an enforced onboard-only observation boundary, an independent referee that no participant can influence, standardized scoring with penalties and rankings, multi-vehicle and team-level evaluation, and reproducible race outcomes. A policy trained under a high-throughput regime of the kind MarineGym provides can be evaluated under the contract Marine Race Arena defines without either system changing; conversely, the failure mode Marine Race Arena exposes in Section 10.6 — policies that cross gates reliably but lose the ordered sequence — is a task-level property that a control-task benchmark is not designed to surface. We regard the two as complementary, and we use MarineGym here as the reference point for what a high-performance underwater RL platform provides, not as a competitor.

2.6. Positioning of Marine Race Arena

Table 1 compares the systems above along criteria that can be checked from their public descriptions: whether the system models underwater physics and sensing, whether it defines a gate-racing task with timing, whether scoring is performed by a component independent of the participant, whether an observation boundary excluding ground-truth pose is part of the contract, whether multiple vehicles are supported, whether a team-level score is defined, and whether a learning-oriented interface is provided. The table is not a ranking. Every underwater simulator listed exceeds Marine Race Arena on dimensions the table does not contain, most obviously hydrodynamic fidelity, sensor variety and, in MarineGym's case, training throughput; Marine Race Arena contributes none of those and depends on a backend for all of them.

The distinction we do claim is narrow and, to the best of our knowledge, not currently occupied: among these systems, Marine Race Arena is the one that combines an underwater racing protocol over ordered gates with an evaluator that is structurally independent of the participant and an observation contract that excludes privileged state from control, and that extends both to team-level evaluation. We do not claim to be the first underwater benchmark, the first underwater multi-robot environment, or a better simulator than any system in the table.

3. Benchmark Model and System Architecture

We specify Marine Race Arena as an *evaluation problem* rather than a controller-design problem: the benchmark fixes the world, the task and the scoring, while a participant supplies only a policy. This section defines the benchmark instance, the participant interface and the runtime architecture that enforce the separation between the two. Everything that follows — the perception front end of Section 5, the controllers of Section 6, the referee of Section 7, the fleet layer of Section 8 and the learned policies of Section 9 — is expressed in the terms introduced here.

3.1. Benchmark Instance

Definition 1 (Benchmark instance). A Marine Race Arena instance is the tuple

$$\mathcal{E} = (\mathcal{W}, \mathcal{G}, \mathcal{S}, \mathcal{F}, C, O, \mathcal{P}, \mathcal{R}), \tag{1}$$

where:

- $\mathcal{W}$ is the *world*: an axis-aligned bounds box $\mathcal{B} = [x_{\min}, x_{\max}]\times[y_{\min}, y_{\max}]\times[z_{\min}, z_{\max}] \subset \mathbb{R}^3$ (with z positive-up, so the navigable volume satisfies $z < 0$) together with the simulation environment selected through the adapter;
- $\mathcal{G} = (g_1, \ldots, g_M)$ is the *ordered* gate sequence; each gate is the tuple $g_k = (c_k, n_k, w_k, h_k)$ of centre $c_k \in \mathbb{R}^3$, passage-direction vector $n_k \in \mathbb{R}^3$ (nonzero, not necessarily unit length) and inner aperture width w_k and height h_k;
- $\mathcal{S}$ is the shared *start default*: a base spawn pose and an optional release policy that a participant inherits only when it does not specify its own;
- $\mathcal{F}$ is the finish condition: completion of the last gate of the last lap (Section 7);
- C is the current field $\mathbf{v}_c : \mathbb{R}^3 \times \mathbb{R}_{\geq 0} \to \mathbb{R}^3$ (Section 4);
- O is the obstacle set;
- $\mathcal{P} = \{1, \ldots, N\}$ is the participant set; participant i carries a vehicle, a sensor profile and a controller, together with an optional spawn pose and release delay $\tau_i \geq 0$ that override the start default of $\mathcal{S}$;
- $\mathcal{R}$ is the referee configuration: gate-validation parameters, penalty values and scoring rules.

Each gate's orientation derives solely from its passage direction n_g, giving a right-handed frame

$$\hat{n} = \frac{n_g}{\|n_g\|}, \quad \hat{r}_g = \frac{\hat{z} \times \hat{n}}{\|\hat{z} \times \hat{n}\|}, \quad \hat{s}_g = \hat{n} \times \hat{r}_g, \tag{2}$$

with world-up $\hat{z} = (0, 0, 1)$; if the passage direction is vertical ($\|\hat{z} \times \hat{n}\| \approx 0$), $\hat{r}_g$ falls back to the world $\hat{x}$ axis so the frame stays well defined. Deriving the frame from the passage direction alone, rather than from a separately authored gate rotation, removes a class of configuration error in which the visual orientation of a gate and the direction in which it must be traversed disagree. The basis is therefore unambiguous and independent of any visual rotation, and it is reused by the beacon placement (Section 4) and by the crossing test (Section 7).

3.2. Participant Interface

Definition 2 (Policy). A participant i implements a policy

$$\pi_i : o_{i,t} \mapsto u_{i,t}, \tag{3}$$

mapping the official observation $o_{i,t}$ (Section 4.6) to a normalized, high-level, body-frame command

$$u_{i,t} = [u^{\text{surge}}, u^{\text{sway}}, u^{\text{heave}}, u^{\text{yaw}}]^\top \in [-1, 1]^4. \tag{4}$$

Table 2: Design requirements for the benchmark layer.

ID	Requirement
R1	Race configuration is declarative and reproducible from a single set of track JSON files.
R2	Official controller observations exclude ground-truth pose, simulator-only state and referee-derived course progress.
R3	Referee logic is independent of the participant controller and cannot be influenced by it.
R4	Gate dimensions, referee margins and track geometry are not changed to hide failures.
R5	Every run emits structured event logs and machine-readable summaries.
R6	Single-vehicle behaviour is unchanged when fleet mode is disabled.
R7	Fleet mode aggregates a team score while preserving per-vehicle diagnostics.

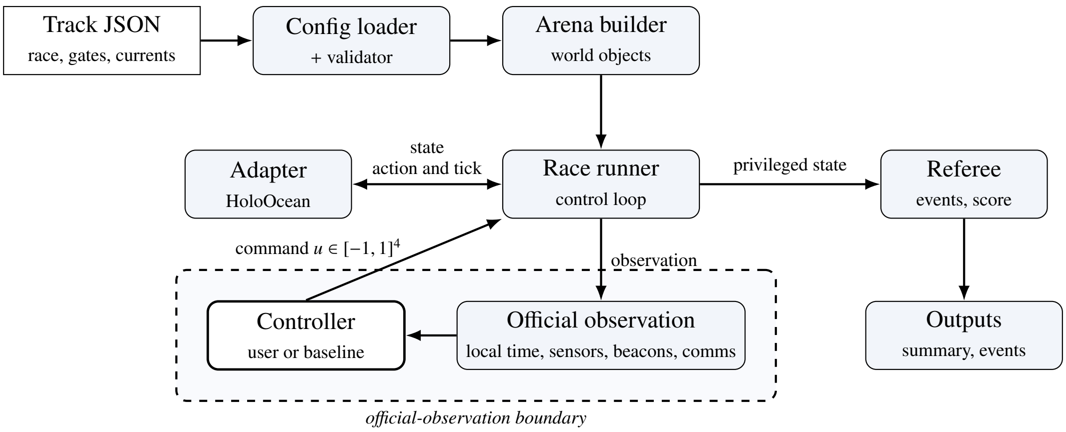

Figure 2: Runtime architecture and official-observation boundary between the controller and referee.

The action of (4) is the portable, constrained interface that an adapter maps to the selected vehicle. In fleet mode the policy may return $(u_{i,t}, m_{i,t})$, where $m_{i,t}$ is an optional controller-authored communication payload and only $u_{i,t}$ is a physical action. The reference HoloOcean adapter also accepts a direct eight-thruster `thrusters` action for its BlueROV2-class agent. Time is discretized at a fixed control timestep Δt, with $t_k = k\,\Delta t$. The adapter clamps $u_{i,t}$ to actuator limits and applies depth-bound safety before mapping it to the vehicle; in official mode the policy reads only the official observation $o_{i,t}$, whose contract excludes privileged state.

Fixing the action space at four normalized body-frame channels is what allows a rule-based controller, a hybrid controller and a neural policy to be compared without any of them gaining actuator authority the others lack. The learned policies of Section 9 emit exactly the vector of (4) and are subject to the same clamping and depth-safety rules as the reference controllers.

3.3. *Design Requirements*

The design follows the seven requirements in Table 2. They encode two commitments. First, scoring logic is independent of the participant and cannot be manipulated through privileged feedback. Second, enabling fleet mode must not silently change single-vehicle behaviour, so that a fleet result and a single-vehicle result remain comparable.

3.4. *Runtime Architecture*

The runtime separates scenario construction, vehicle autonomy and race evaluation. A declarative track configuration defines the environment, the ordered gates, the beacon network, the disturbances, the participants and the scoring rules; after validation it becomes an arena instance connected to a simulator adapter. The adapter isolates sensing, actuation and time advancement, so the controller and referee interfaces are reusable across simulation backends. An engine-free adapter supports deterministic unit testing and is explicitly excluded from every reported result: each experimental run records whether the fallback was permitted, and all results in this paper were produced with the fallback disabled and the real HoloOcean adapter confirmed in the manifest.

During execution the race runner advances the simulator and mediates vehicle–environment interaction. The controller receives an observation containing only participant-local time, allow-listed onboard measurements, physically received beacon packets and optional inter-vehicle messages. It returns a bounded motion command and cannot access simulator state, gate geometry, race progress or referee decisions, so navigation and course progression must be inferred locally. In parallel, the referee uses privileged simulator state to validate crossings and rule violations and to produce event logs, participant results and team-level scores. Referee-derived targets, progress and status are never returned to the controller; disagreements with the controller's local estimate are recorded for offline analysis rather than corrected, and Section 7.7 quantifies how large those disagreements actually are. Fig. 2 summarizes the separation and the controller-visible observation boundary.

4. Vehicle, Simulator, Sensing and the Observation Contract

This section describes the simulation backend, the vehicle and its sensors, the acoustic and current models, the declarative track configuration and the official observation contract that is enforced independently of the referee.

4.1. Simulator and Vehicle

The benchmark architecture is not tied to a particular vehicle: adapters provide simulator state, filtered sensing and action mapping to the common race and referee layers. The reference adapter used in every HoloOcean experiment reported here drives HoloOcean [9] with a BlueROV2-class agent [11] in direct eight-thruster mode. The simulator advances at 30 physics ticks per second.

Two control timesteps appear in this paper and are never pooled. The frozen benchmark matrix of Section 10.1 uses $\Delta t = 0.033$ s, so each control step corresponds to one physics tick and the effective control rate is ≈ 30 Hz. The current-free completion demonstration (Section 10.2) and the matched learned-controller benchmark (Section 10.6) use $\Delta t = 0.1$ s, i.e. 10 Hz control over the same 30 Hz physics. The slower rate reduces the number of policy evaluations per simulated second, which matters when a neural policy is in the loop, and it produces systematically longer completion times for the same controller on the same circuit. Every table in this paper states the control rate of the runs it reports, and no time from one rate is compared against a time from the other.

In the reference HoloOcean adapter, a high-level command $u = [u^{\text{surge}}, u^{\text{sway}}, u^{\text{heave}}, u^{\text{yaw}}]^\top$ is mapped to the eight BlueROV2 thrusters by a fixed mixing matrix. The four vertical thrusters receive u^{heave}; the four horizontal thrusters receive

$$\tau_{1\text{-}4} = s_\tau \begin{bmatrix} u^{\text{surge}} + u^{\text{sway}} + \kappa\, u^{\text{yaw}} \\ u^{\text{surge}} - u^{\text{sway}} - \kappa\, u^{\text{yaw}} \\ u^{\text{surge}} + u^{\text{sway}} - \kappa\, u^{\text{yaw}} \\ u^{\text{surge}} - u^{\text{sway}} + \kappa\, u^{\text{yaw}} \end{bmatrix}, \tag{5}$$

where $\kappa = 0.35$ is the yaw-mixing gain and s_τ the thruster scale; each component is clamped to the thruster limit. A controller may instead return the eight thruster values directly. The high-level interface is the default because it is simpler and vehicle-agnostic: a controller expresses intended body-frame motion while the framework handles mixing and depth safety. Direct thruster control remains available when finer actuator authority is required, but no result in this paper uses it.

4.2. Sensor Suite

Table 3 lists the onboard sensors. The official profile combines a forward RGB camera, a depth sensor, an IMU and a DVL; an onboard contact flag is also permitted when configured. Sensors providing world-frame velocity and ground-truth pose, location, rotation and dynamics exist internally for simulation and referee evaluation, but are rejected at the adapter boundary and cannot appear in the official controller observation (Section 4.6).

Table 3: Onboard sensor suite of the BlueROV2.

Sensor	Rate	Output and configuration
Depth	30 Hz	Scalar depth (m); noise $\sigma = 0$.
IMU	30 Hz	Linear acceleration and angular rate, with bias terms.
DVL	15 Hz	Body-frame velocity (m/s); beam elevation 22.5°, max range 50 m.
Front camera	30 Hz	RGB image, 640×480, 90° field of view.
Collision	optional	Boolean onboard contact flag when configured.

World-frame velocity, pose-derived heading, pose-derived depth and environmental-current telemetry are not controller-visible in official mode. Pose and velocity sensors may exist internally for simulation and referee evaluation only.

4.3. Acoustic Beacon Model

Let $(\hat{n}, \hat{r}_g, \hat{s}_g)$ be the right-handed gate frame of (2), with $\hat{n}$ the passage normal and $\hat{s}_g$ the gate-up axis. Each gate carries one beacon, placed at $b_g = c_g + \delta_n \hat{n} + \delta_r \hat{r}_g + \delta_s \hat{s}_g$ relative to the gate centre c_g (default offset $(0, 0, 0.35)$ m along $\hat{s}_g$). For a receiver at position p_r with heading ψ, let $\boldsymbol{\delta} = b_g - p_r$, let $\rho = \|\boldsymbol{\delta}\|$ be the geometric range and let $\tilde{\rho}$ be its noisy delivered measurement. The beacon reports $\tilde{\rho}$, a yaw-relative bearing, an elevation and a signal strength:

$$\rho = \|\boldsymbol{\delta}\|, \qquad \tilde{\rho} = \max(0, \rho + \eta_\rho), \qquad \eta_\rho \sim \mathcal{N}(0, \sigma_\rho^2), \tag{6}$$

$$\beta = \mathrm{wrap}(\,\mathrm{atan2}(\delta_y, \delta_x) - \psi) + \eta_\beta, \tag{7}$$

$$\varepsilon = \mathrm{atan2}(\delta_z,\ \sqrt{\delta_x^2 + \delta_y^2}) + \eta_\varepsilon, \tag{8}$$

$$\mathrm{SS} = \max\Big(0,\ 1 - \tfrac{\tilde{\rho}}{\rho_{\max}}\Big),$$

where $\rho_{\max}$ is the beacon range and wrap($\cdot$) maps to $(-180°, 180°]$. Bearing and elevation carry independent angular perturbations $\eta_\beta, \eta_\varepsilon \sim \mathcal{N}(0, \sigma_\theta^2)$ with σ_θ in degrees, and the range carries a single perturbation η_ρ with σ_ρ in metres. Each is drawn once per packet, and the signal strength derives from the same noisy range $\tilde{\rho}$, so no cleaner distance leaks back as a side channel. A transmission is not received beyond $\rho_{\max}$ or when a Bernoulli dropout fires. The official tracks set σ_θ and σ_ρ to a common per-track value in $\{0.2, 0.45, 0.6\}$ (degrees and metres respectively) and dropout probabilities up to 0.04.

Every ordered gate has one independent periodic transmitter with a unique contiguous identifier B01, ..., BM. All beacons transmit regardless of referee state; at each update a vehicle receives every in-range, non-dropped packet. The framework does not tell the controller which packet corresponds to its next gate, and does not suppress the others: deciding which received identifier to pursue is part of the participant's autonomy (Sections 5 and 6). Packet randomness is seeded by the run seed, the transmitter, the receiver and the transmission index, which makes reception independent of participant count and call order and therefore keeps a fleet run reproducible.

4.4. Current Field Model

The current field is a sum of analytic primitives evaluated at the vehicle position p and time t,

$$\mathbf{v}_c(p, t) = \sum_j \mathbf{v}_j(p, t), \tag{9}$$

where each primitive is one of the following. A *constant* term contributes a fixed velocity $\mathbf{v}_0$. A *localized jet* centred at c_j with radius R_j contributes, for $\|p - c_j\| \le R_j$,

$$\mathbf{v}_j = \mathbf{v}_0\, \omega(d_j), \qquad \omega(s) = \begin{cases} \max(0, 1 - \frac{s}{R_j}) & \text{linear,} \\ \exp(-\frac{s^2}{2\sigma_j^2}) & \text{Gaussian,} \end{cases} \tag{10}$$

where $d_j = \|p - c_j\|$ is the jet distance and $\sigma_j = \max(R_j/2, 10^{-6})$ the jet width; the contribution is zero outside R_j. A *sinusoidal* term modulates one axis, $v_{j,\mathrm{axis}}(t) = v_{0,\mathrm{axis}} + A \sin(2\pi f t + \phi)$. A *vortex* of radius R_j, tangential speed V_t and vertical speed V_z contributes, with $r = \sqrt{\Delta x^2 + \Delta y^2}$ and tangent $\hat{\theta} = (-\Delta y, \Delta x)/r$,

$$\mathbf{v}_j = (\hat{\theta}_x\, V_t\, \omega(r),\ \hat{\theta}_y\, V_t\, \omega(r),\ V_z\, \omega(r)). \tag{11}$$

The provided `strong` profile superposes a constant set-flow of $[0.75, 1.05, 0]$ m/s, two Gaussian jets, a vortex and a vertical sinusoid; the `medium` profile scales these magnitudes by 0.5. Both named profiles are defined relative to the gate layout and are available on each official circuit. The HoloOcean adapter evaluates the field at every vehicle before each control step. Crucially, the current field is applied physically and is *not* reported to the controller: a participant must infer the disturbance from its own onboard measurements, in practice from the difference between commanded motion and body-frame DVL velocity.

4.5. Declarative Track Configuration

A track is a self-contained JSON file. Its principal sections are `race` (name, timing mode, laps and duration), `benchmark_task`, `world` (bounds and environment), `start` and `finish`, `track` and `gates` (the ordered gate sequence with centres, passage directions and aperture sizes), `beacon`, `currents` and `current_profiles`, `obstacle_generation` and `obstacles`, `participants` (vehicle, spawn, sensors, controller and optional `start_delay_s`) and `referee` (gate validation, penalties and scoring). The named `benchmark_task` (`clean_gate`, `obstacle_gate`, `current_gate` or `multi_rov`) selects a validation contract. Listing 1 shows a representative excerpt.

Listing 1: Representative excerpt of a track configuration.

```
"track": {
  "gate_inner_size_m": [1.5, 1.5],
  "gate_sequence": ["G01", "G02", "..."]
},
"gates": [
  { "id": "G01", "position": [4.0, -2.0, -3.5],
    "rotation_rpy_deg": [0.0, 0.0, 0.0],
    "passage_direction": [1.0, 0.0, 0.0] }
],
"beacon": {
  "range_m": 90.0,
  "angular_noise_std_deg": 0.2,
  "range_noise_std_m": 0.2,
  "dropout_probability": 0.0,
  "update_rate_hz": 10.0
},
"participants": [
  { "id": "bluerov2_01", "vehicle": "BlueROV2",
    "controller": "rule_gate_baseline",
    "start_delay_s": 0.0 }
],
"referee": {
  "penalties": { "minor_collision_s": 5.0 },
  "gate_validation": {
    "vehicle_clearance_margin_m": 0.1 }
}
```

The three official circuits are summarized in Table 4 and their layouts shown in Fig. 3. They are deliberately heterogeneous: Horseshoe Bay is a short, largely planar baseline; Vertical Serpent forces repeated vertical transitions and slalom sequencing; Mixed Endurance is more than twice as long as Horseshoe Bay and combines both. All three use $1.5\,\mathrm{m} \times 1.5\,\mathrm{m}$ apertures, and neither the aperture size nor the referee margin was changed at any point of this study (requirement R4).

The `obstacle_gate` task accepts explicit fixed obstacles or deterministic random boxes generated between consecutive gates with a configured density and minimum clearance. The HoloOcean adapter records the requested and physically spawned counts, and an obstacle construction check is accepted only when they match. This verifies that obstacles can be instantiated on a circuit; it is deliberately separate from any claim that a reference controller avoids them, and no obstacle-avoidance result is reported in this paper.

Table 4: Official track configurations.

Track file	Track name	Gates	Laps	Total gates	Length (m)	Intended use
`marine_race_horseshoe_bay.json`	Marine Race Horseshoe Bay	12	1	12	93.8	Clean-gate horseshoe baseline
`marine_race_vertical_serpent.json`	Marine Race Vertical Serpent	17	1	17	118.3	Vertical and slalom gate sequencing
`marine_race_mixed_endurance.json`	Marine Race Mixed Endurance	22	1	22	206.3	Longer endurance and current-oriented layout

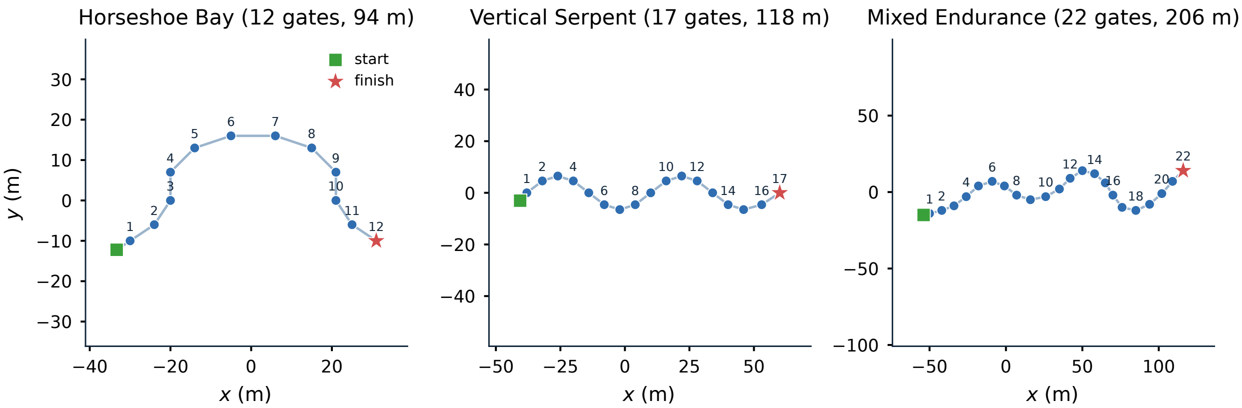

Figure 3: Top-down (x, y) layouts of the three official circuits, drawn from the released track JSON: ordered gates, start (square) and finish (star). The circuits vary in length (93.8, 118.3 and 206.3 m), in gate count (12, 17 and 22) and in vertical structure.

4.6. *Official Observation Contract*

The controller `step` function receives a dictionary with the fields in Table 5. The builder is structurally independent of the referee: its signature contains the track configuration, the arena, the adapter and the participant state, but no referee object, so referee-derived quantities are not merely omitted by convention but are unreachable from the code path that constructs the observation. The builder computes participant-local elapsed time, requests only allow-listed onboard sensors from the adapter and queries the beacon field for physically received periodic transmissions. Numeric arrays are made JSON-safe while image arrays are preserved unchanged. When the optional inter-vehicle acoustic channel is enabled (Section 8), the observation additionally carries a `comms` inbox whose payloads are authored by another controller from its own legal information.

Three quantities are worth naming explicitly because their absence defines the task. The controller never receives the global vehicle pose, so it cannot localize itself in the world frame. It never receives gate positions, gate normals or aperture dimensions from the simulator, so the geometry of the course must be inferred from beacons and images. And it never receives the referee's notion of which gate is next, whether the previous crossing was valid, or how many gates remain, so course progression is a controller-side estimate that can be, and sometimes is, wrong.

5. Onboard Gate Perception and Target Association

A racing benchmark is only as strict as its weakest information leak. The observation contract of Section 4.6 withholds gate geometry and referee progress, but it delivers a camera image that may contain several gates and a list of acoustic packets whose identifiers carry no ordering information. Deciding *which* visible rectangle is the gate the vehicle must pass next is therefore not a preprocessing step that the framework can perform on the participant's behalf: doing so would silently reintroduce course knowledge into the control loop. This section describes the onboard front end that the reference controllers use to make that decision, states the assumptions it depends on, and reports what it achieves and where it fails.

Table 5: Top-level fields of the official observation.

Field	Meaning
`local_time_s`	Elapsed time since this vehicle's release; it is not official race time.
`sensors`	Filtered onboard dictionary containing only the configured camera, depth, IMU, DVL and optional collision measurements.
`beacons`	List of physically received packets from independent transmitters; it may be empty or contain several identifiers, and no field marks which one is the next gate.
`comms`	Present only when the optional inter-vehicle acoustic channel is enabled: an `inbox` of messages received from teammates, each with the sender identifier, the controller-authored payload and a receiver-local reception timestamp (Section 8).

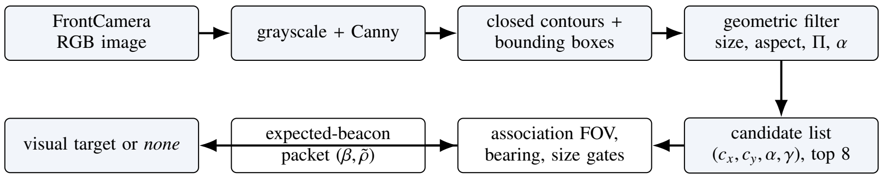

Figure 4: The onboard gate-perception front end. Camera pixels produce an unlabelled candidate list; the controller's own expected-beacon packet supplies the bearing that selects among them. No simulator gate pose, global vehicle pose or referee target enters any stage.

5.1. *Gate Detection from the Forward Camera*

Motivation. The gates are rendered as square frames of light-coloured bars. A naive bright-pixel segmentation is unusable in this environment: HoloOcean tints the public `white` prop material strongly with the water column, so a gate bar can be *darker* than the surrounding water and the seabed, and a generic brightness or saturation mask selects most of the scene. The detector therefore keys on the one property that survives the tint, namely that a gate projects to a near-rectangular closed contour of bounded aspect ratio, and validates candidates geometrically rather than photometrically.

Forward process. Given a FrontCamera image of width W and height H, the detector proceeds as follows.

1. **Preprocessing.** The RGB frame is normalized to 8-bit, converted to grayscale and filtered with a Canny edge detector with hysteresis thresholds $(30, 80)$.
2. **Contour extraction.** Closed contours are extracted from the edge map, and each contour is reduced to its axis-aligned bounding box $(x, y, w_{\mathrm{px}}, h_{\mathrm{px}})$.
3. **Geometric filtering.** A candidate survives only if it is large enough to be a gate and shaped like one: $w_{\mathrm{px}} \geq \max(12, 0.05W)$ and $h_{\mathrm{px}} \geq \max(12, 0.05H)$; the box occupies less than 0.92 of either image dimension, which rejects frame-filling background structure; the aspect ratio $a = w_{\mathrm{px}}/h_{\mathrm{px}}$ lies in $[0.45, 2.40]$; the bounded perimeter ratio $\Pi = L_{\mathrm{contour}}/(2(w_{\mathrm{px}} + h_{\mathrm{px}}))$ lies in $[0.50, 2.20]$, which rejects filled background regions and irregular surface reflections whose contour length is inconsistent with their bounding box; and the area fraction $\alpha = w_{\mathrm{px}}h_{\mathrm{px}}/(WH)$ is at least 0.008.
4. **Normalized image geometry.** A surviving candidate is reported in image-normalized coordinates centred on the optical axis,
$$c_x = \frac{(x + \frac{1}{2}w_{\mathrm{px}}) - \frac{1}{2}W}{\frac{1}{2}W}, \qquad c_y = \frac{(y + \frac{1}{2}h_{\mathrm{px}}) - \frac{1}{2}H}{\frac{1}{2}H}, \tag{12}$$
both clamped to $[-1, 1]$, together with the area fraction α and the width and height fractions w_{px}/W and h_{px}/H. The pair (c_x, c_y) is the normalized image error with respect to the camera centre, and is the quantity the visual servo drives to zero.
5. **Confidence.** Each candidate carries a bounded confidence combining four normalized scores — rectangularity, aspect plausibility, apparent size and centredness,
$$\gamma = 0.38\, s_\Pi + 0.24\, s_a + 0.20\, s_\alpha + 0.18\, s_c \in [0, 1], \tag{13}$$

with $s_\Pi = 1 - |\Pi - 1|/1.20$, $s_a = 1 - |\log a|/\log 3$, $s_\alpha = \alpha/0.10$ and $s_c = 1 - 0.45|c_x| - 0.30|c_y|$, each clamped to $[0, 1]$. Candidates with $\gamma < 0.38$ are discarded.

6. **Multiple candidates.** Surviving candidates are sorted by confidence, deduplicated when their normalized centres differ by less than 0.04 in both axes, and the strongest eight are returned. A parallel coarse-grid colour-blob path runs alongside the contour path and contributes additional candidates, which keeps the detector usable in minimal installations where OpenCV is unavailable.

Technical advantage. Reporting a *list* of candidates with per-candidate geometry, rather than a single best detection, is what makes the next stage possible. A detector that silently returns its most confident rectangle would resolve the association question implicitly, and would resolve it wrongly whenever the most confident rectangle belongs to a later gate — which is common on the official circuits, where consecutive gates are frequently visible together.

5.2. Acoustic–Visual Target Association

Motivation. The controller knows which beacon identifier it is currently pursuing, because it maintains that state itself (Section 6), and it receives a noisy bearing β and range $\tilde{\rho}$ for that identifier from (6). It does not know which image rectangle that beacon belongs to. Association is the step that binds the two modalities, and it is also the step where an incorrect commitment is most expensive: locking onto the wrong gate produces a confident, well-servoed approach to a gate the referee will score as a miss.

Forward process. Let $\mathcal{T}$ be the candidate list, and let β and $\tilde{\rho}$ be the bearing and range of the expected beacon, or $\varnothing$ when no packet for that identifier was received.

1. **Field-of-view gate.** If $|\beta| > 70°$ the association returns nothing. With a 90° horizontal field of view, a target that far off the nose cannot be the centre of the expected gate. This case matters immediately after a passage, when the just-cleared beacon lies behind the vehicle while the next gate is already visible ahead; accepting the forward rectangle there would change target identity without the tracker ever deciding to advance.
2. **Implied bearing.** Each candidate is assigned the body bearing implied by its horizontal image position,

$$\hat{\beta}(c_x) = -\arctan(c_x), \tag{14}$$

   in degrees, with positive bearing to camera-left, and a mismatch $m = |\hat{\beta}(c_x) - \beta|$.
3. **Near-field size gate.** When $\tilde{\rho} < 1.6\,\text{m}$, a 1.5 m aperture must subtend a substantial part of the image, so candidates are kept only if $\alpha \geq 0.05$ or either linear fraction is at least 0.30. A small, complete rectangle at that acoustic range is a later gate, not a reappearance of the current one.
4. **Conflict rejection.** If any candidate satisfies $m \leq 32°$, every candidate with $m > 32°$ is rejected outright. The acoustic bearing is thus a hard identity gate whenever it is informative, not merely a soft score term, which prevents a large blob spanning several visible gates from outscoring the correct one.
5. **Scoring.** Surviving candidates are ranked by confidence plus a centredness term and a size term, minus a bearing penalty $0.050\,m$. The size term is deliberately dominated by a *relative* component, $s_{\text{rel}} = \sqrt{\alpha/\alpha_{\max}}$ where $\alpha_{\max}$ is the largest area fraction in the list, because an absolute size score saturates when two gates are both reasonably large and then lets the more centred, farther gate win. When $|\beta| > 25°$ an additional side test is applied: the candidate must lie on the image side the acoustic bearing indicates, by at least 0.10 normalized units (0.18 beyond 45°).
6. **Fallback.** With no expected-beacon bearing available, the association degrades to a bearing-free default that prefers large, confident, centred candidates. With no surviving candidate at all, the front end reports that no visual target is available for this frame, and the controller must proceed on acoustic and inertial evidence alone.

Technical advantage. Because association is driven by the controller's own expected identifier and by a physically received bearing, the front end never needs, and never receives, a ground-truth answer. Nothing in the pipeline consults simulator gate poses, the global vehicle pose, or the referee's target index. The consequence is that a wrong association is a genuine autonomy failure that the referee will score, which is exactly the property a benchmark needs.

5.3. *An Exploratory Pose-Aware Extension*

Centre-only vision is geometrically ambiguous. A gate centred in the image but approached obliquely, and a gate approached head-on, produce similar centroids; so do lateral displacement with a frontal gate plane and yaw misalignment with a centred gate. We therefore implemented an exploratory pose-aware front end that attempts to recover the gate-plane pose from the same camera image and the publicly documented aperture size, and we report it here as an extension of the perception stack rather than as a component of the main results.

The extension thresholds the bright bars, extracts the frame's inner hole with a hierarchical contour retrieval, approximates it by a quadrilateral, validates the four corners for convexity, area, aspect, side-length consistency and non-self-intersection, and orders them canonically. Camera intrinsics follow from the configured 640×480 sensor and 90° horizontal field of view, giving $f_x = f_y = W/(2\tan(\mathrm{FOV}/2)) = 320$ and $(c_x, c_y) = (320, 240)$ in pixels. Planar PnP on the four corners against the known square model yields up to two solutions; the square-planar sign ambiguity is resolved using the bar-height ratio, which side bar appears taller, together with the reprojection error. When PnP is unavailable or unstable, a projective stage still reports a frontal / rotated-left / rotated-right class from the bar-height ratio and skew, and a known-size distance estimate $z = f_y \cdot 1.5/h_{\mathrm{aperture,px}}$ supplies a metric-like translation. A controller-side exponential filter with angular wrapping and staleness expiry stabilizes the single-frame estimate. The resulting twelve features — pose availability, normalized lateral, vertical and forward offsets, sine and cosine of plane yaw and pitch, normalized reprojection error, apparent width and height fractions and quadrilateral skew — append to the observation encoding without displacing any existing feature, and are masked when unavailable rather than filled with ground truth.

Measured limits. We evaluated this extension on a dedicated rig in real HoloOcean in which the gate plane was rotated by 0°, ±25° and ±45° relative to the camera, with 2.0 m apertures, over 60 frames in stationary and moving conditions. The result is negative and we state it plainly: a pose was available in only 11.7% of frames, the median plane-yaw error was 48.2° with a 90th percentile of 50.3°, and both the orientation-class accuracy and the yaw-sign accuracy were 0.00. The median distance error was 1.73 m. Under strong obliquity the sign disambiguation fails, and the projective fallback reports a frontal plane for gates rotated by 45°.

We therefore do not use pose-aware features to support any experimental claim in this paper. The reference controllers of Section 6 and the learned policies of Section 9 both operate on the centre-only visual features of Section 5.1, and the pose extension is reported as an implemented but unvalidated component and as an explicit limitation (Section 12). Its value in the present work is diagnostic: it identifies strong gate obliquity as the regime in which monocular geometric reasoning about this gate model breaks down, which is information a benchmark should surface rather than conceal.

5.4. *Perception Audit on the Official Circuits*

The oblique rig above is an adversarial probe, not the operating regime. To characterize the front end where the controller actually uses it, we instrumented the perception stack along the reference controller's own trajectory on all three official circuits in real HoloOcean, recording 60 frames per circuit (seeds 67000–67002). Ground truth was used only offline to score the detector and never entered the control loop. Table 6 reports the result, and Fig. 5 shows three representative states.

Three observations follow. First, detection is available in 88.3–91.7% of frames and a valid four-corner aperture in 78.3–83.3%, so the front end is usable but intermittent; the controller must tolerate gaps rather than assume a detection every frame. Second, metric PnP succeeds in only about half of frames (48.3–53.3%), which is consistent with the negative pose result above and is why the projective fallback matters. Third, and most relevant to the association design, the target chosen online agreed with an offline referee selection in every frame on all three circuits, the multi-candidate association was correct in every multi-candidate frame (7, 4 and 11 frames respectively), and no spurious online target switch occurred. Under the near-frontal viewing geometry that the centre-then-commit approach maintains, the plane-orientation estimate is also accurate (median error 0.018–1.129°, no error above 30°, no sign error, no frame-to-frame yaw jump above 20°).

The contrast between this table and the oblique rig is the substantive finding. The pose front end is not uniformly weak; it is accurate in the near-frontal regime and fails under strong obliquity. Because the reference controller actively maintains a near-frontal approach, it operates inside the regime where the front end works, which explains

Table 6: Onboard perception audit along the reference controller's own trajectory on the three official circuits, real HoloOcean, 60 frames per circuit. Ground truth was used only offline to score the front end and never entered the control loop.

	Availability			Plane orientation error			Association	
Circuit	Det.	4-corner	PnP	med. (°)	p90 (°)	> 30°	multi	correct
Horseshoe Bay	0.900	0.817	0.517	0.018	1.204	0.000	7	1.000
Vertical Serpent	0.883	0.783	0.483	1.129	4.287	0.000	4	1.000
Mixed Endurance	0.917	0.833	0.533	0.064	0.064	0.000	11	1.000

Det.: fraction of frames with at least one accepted detection. *4-corner*: fraction with a validated aperture quadrilateral. *PnP*: fraction with a metric planar-PnP solution. *multi*: number of frames in which more than one candidate was present. *correct*: fraction of multi-candidate frames whose online association agreed with an offline referee selection within a normalized centre tolerance of 0.25; the online target and the offline selection agreed on every frame of all three circuits. No circuit recorded a wrong orientation sign, a gross orientation error above 30°, a frame-to-frame yaw jump above 20°, or an unintended online target switch. On Mixed Endurance the expected gate was outside the field of view in 3 of the 60 frames; the in-view detection and four-corner rates there are 0.912 and 0.825. This is a characterization of the perception component from a diagnostic capture, not a controller performance claim.

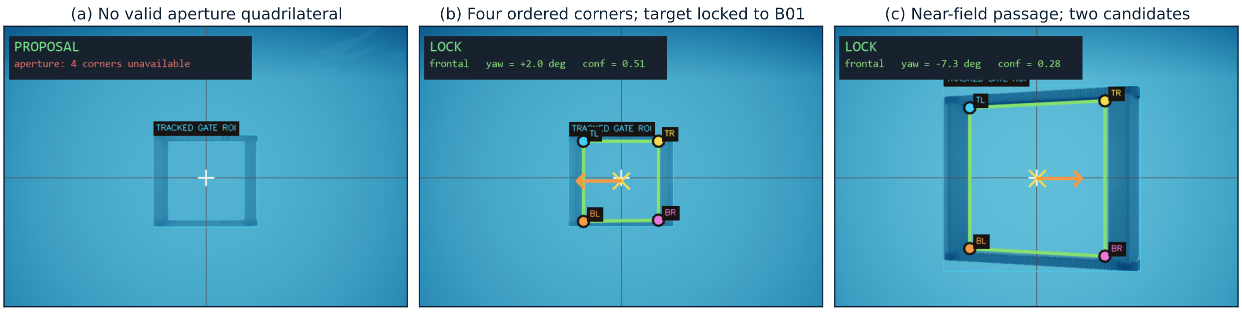

Figure 5: Onboard gate perception in real HoloOcean, one representative state per official circuit. Each panel shows the forward-camera frame with the detector's own overlays: the tracked aperture region, the four canonically ordered corners (TL, TR, BR, BL), the image-centre crosshair, the estimated gate centre and the image error vector of (12) that the visual servo drives to zero. (a) Mixed Endurance: a target is detected but no valid aperture quadrilateral is available, so the front end degrades to the centre-only estimate and reports an unknown orientation. (b) Horseshoe Bay: four validated corners with the target associated to the expected beacon `B01` at 4.39 m and −0.0° bearing, tracker phase VISUAL_ALIGN. (c) Vertical Serpent: a near-field passage at 2.25 m with two raw candidates, in which the association of Section 5.2 retains the correct target, tracker phase VERIFY_EXIT. Nothing shown here is derived from simulator gate poses or from the referee. Panel chrome from the interactive viewer has been cropped and its state badge redrawn in English; the detector overlays and all quoted values are unmodified.

both why the rule-based baselines complete circuits and why the pose features are not needed to do so. This audit is a characterization of the perception component and, being based on 60 frames per circuit from a diagnostic capture, is reported as such; no completion claim in Section 10 rests on it.

6. Controller Interface and Rule-Based Reference Controllers

A central usability goal of Marine Race Arena is that an autonomy stack can be integrated through a small interface without modifying the runner, the referee or the adapter. This section specifies that interface, the dynamic loading mechanism, the controller-local course tracker and the two rule-based reference controllers whose comparison anchors the evaluation.

6.1. Controller Interface

A controller implements three methods: `reset(mission_info)`, called once before the run with a minimal static assignment; `step(observation)`, which receives the official observation of (3) at every control tick and returns the command of (4); and `close()`, called at teardown. The contract is duck-typed, so any object exposing the three callables is accepted and a participant need not depend on framework base classes. A manual controller may raise a dedicated stop exception from `step` to end a run cleanly; any other exception is recorded as a controller error and terminates the participant. Listing 2 shows a minimal example.

Table 7: High-level controller command fields of (4).

Field	Interpretation
`surge`	Longitudinal body-frame thrust; positive moves forward.
`sway`	Lateral body-frame thrust; sign follows the adapter mixing of (5).
`heave`	Vertical thrust; additionally constrained by depth-safety logic near the world bounds.
`yaw`	Yaw-rate command; sign follows the adapter mixing of (5).

Listing 2: Minimal custom controller.

```
class MyController:
    def reset(self, mission_info):
        self.tracker = LocalCourseTracker(
            initial_beacon_id=mission_info["initial_beacon_id"],
            total_beacons=mission_info["total_beacons"],
            laps=mission_info["laps"])

    def step(self, observation):
        sensors = observation["sensors"]
        state = self.tracker.update(
            local_time_s=observation["local_time_s"],
            beacons=observation["beacons"],
            camera_image=sensors.get("FrontCamera"),
            dvl_velocity=sensors.get("DVLSensor"))
        if state.finished:
            return {"surge": 0.0, "sway": 0.0, "heave": 0.0, "yaw": 0.0}
        # ... map state.beacon and camera to a command ...
        return {"surge": 0.3, "sway": 0.0, "heave": 0.0, "yaw": 0.0}

    def close(self):
        pass
```

The `mission_info` passed to `reset` is deliberately minimal: the initial beacon identifier, the number of beacons and the lap count. It contains no gate positions, no course length in metres and no ordering beyond the starting identifier. The high-level command fields are listed in Table 7; values are normalized to $[-1, 1]$ and clamped by the framework before the action mapping of (5). In fleet mode a controller may additionally return a small `message` payload alongside its command and read incoming teammate messages from the observation.

6.2. Dynamic Loading

A controller is selected at run time in one of three ways: a built-in alias such as `rule_gate_baseline` or `rule_gate_center_then_commit`; a dotted module path with an explicit class; or a file path together with `-controller-class`, imported dynamically. For inspection and data collection the runner can also be launched with the manual `keyboard` or `pygame` controllers. An external policy is therefore evaluated without modifying the package:

```
python -m marine_race_arena.scripts.run_marine_race \
  --track .../marine_race_horseshoe_bay.json \
  --benchmark-task clean_gate \
  --controller path/to/my_controller.py \
  --controller-class MyController \
  --adapter holoocean --official --headless
```

The same mechanism loads the learned policies of Section 9: a thin wrapper exposes the three interface methods, encodes the official observation into the policy's input vector and returns the policy's output as the command of (4). No part of the runner, referee or adapter is aware that the controller is neural.

6.3. *Controller-Local Course Progression*

Motivation. Because the observation carries no course progress (Section 4.6), a controller that must pass M gates in order has to maintain that ordering itself. The framework supplies a reusable component, the `LocalCourseTracker`, that both reference controllers own; the runner supplies no progression signal of any kind.

Module design. The tracker initializes from the public mission assignment `initial_beacon_id` and maintains a local expected beacon identifier, a completed count, a lap index and a status through the sequence

$$\text{SEARCH} \rightarrow \text{APPROACH} \rightarrow \text{VISUAL\_ALIGN}$$
$$\rightarrow \text{COMMIT} \rightarrow \text{VERIFY\_EXIT} \rightarrow \text{ADVANCE},$$

ending in FINISHED after the final locally confirmed beacon. At each tick it filters the received packet list for its own expected identifier — packets from other transmitters are neither labelled nor selected by the framework — and combines that packet with the associated visual target of Section 5.2 and with body-frame DVL displacement.

Advancement conditions. The tracker enters and completes a passage only when persistent camera, acoustic and DVL evidence agree with the expected beacon. Five conditions individually block an advancement: a temporary visual loss, a missing acoustic packet, an isolated range jump, a range rise without a prior close approach, and a commit without forward DVL displacement. Incomplete or inconsistent evidence returns the tracker to local acquisition without advancing. After each non-final advancement it holds an exit-clearance phase; after the final advancement it enters FINISHED directly and commands zero motion. Fig. 6 shows the state machine.

Technical advantage and its cost. Requiring agreement across three modalities is what keeps the local estimate close to the referee's: over the frozen benchmark matrix the tracker produced 1472 advancements against the referee's 1474, with 11 false and 7 missed (Section 7.7). The cost is latency — the tracker confirms a passage a median of 0.858 s after the referee records it — and this lag is visible in the controller's behaviour rather than hidden, because the exit-clearance phase begins only once the tracker is convinced. Controller-local snapshots and independent referee crossings are logged separately for offline comparison and are never fed back into control.

The two reference controllers and their local-progression logic were designed and validated for the gate geometry, beacon placement, sensing configuration and vehicle dynamics of the three official circuits used in this study; they are interpretable baselines, not tuned optimal policies, and we make no claim that their thresholds transfer unchanged to a different course family.

6.4. *Continuous Servo*

Both reference controllers are deterministic and interpretable rule-based baselines rather than optimized policies, and both use exactly the same onboard inputs: received acoustic packets, the forward RGB camera, depth and IMU measurements and body-frame DVL velocity. They combine acoustic guidance, visual centring, depth hold and DVL-based passage verification through the shared tracker of Fig. 6.

Continuous Servo steers toward the expected beacon using its bearing and elevation, and refines that alignment with the associated visual target as the gate becomes visible. Depth feedback maintains the vertical reference and DVL motion provides the evidence that the vehicle has crossed and cleared the gate. Its defining behaviour appears during COMMIT and VERIFY_EXIT: the controller continues applying bounded visual corrections while moving through the aperture, and therefore keeps servoing toward the observed image centre throughout the passage.

6.5. *Center-then-Commit*

Motivation. Continuous visual correction is well behaved at range and poorly behaved at contact distance. As the vehicle enters the aperture the gate frame leaves the field of view piecewise: bars clip at the image border, the detected contour becomes a partial rectangle, and the centroid of that partial rectangle moves rapidly even though the vehicle is on line. A controller that keeps chasing the centroid then commands large late corrections precisely when the aperture margin is smallest.

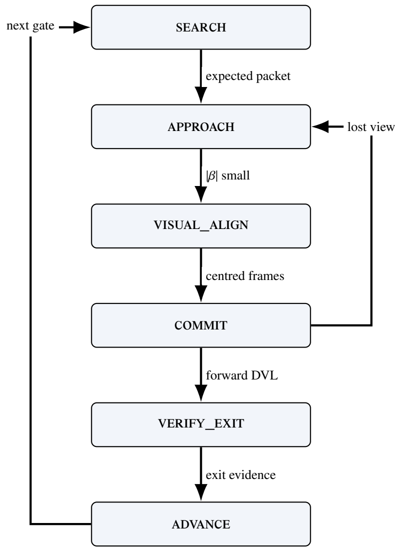

Figure 6: The LocalCourseTracker state machine shared by both reference controllers. advance loops back to search for the next gate and reaches finished after the last locally confirmed passage. No transition consumes referee state.

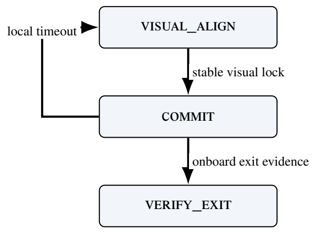

Figure 7: Center-then-commit passage in the commit and verify_exit states, with controller-local confirmation and recovery.

Module design. *Center-then-Commit* retains the same observations, the same acoustic and visual guidance, the same shared tracker and the same local confirmation criterion as Continuous Servo. The only difference is the gate-passage strategy: after establishing a stable visual–acoustic lock, it stops chasing subsequent image-centroid changes during commit and verify_exit, and instead advances through the aperture while holding the locked depth and attitude from onboard depth and IMU feedback. The same DVL-based evidence verifies the passage.

Technical advantage. Because the observations, the tracker and the confirmation logic are unchanged, the matched comparison of Section 10.3 isolates one variable: continuous visual correction versus a stabilized committed passage. This is the cleanest ablation the benchmark supports, and Section 10.3 shows that its outcome is condition-dependent rather than uniform — the staged passage costs a little speed on a short clean circuit and buys substantial reliability on a long one and under disturbance.

7. Independent Referee and the Information Boundary

The referee is the component that makes results comparable, and it is the component a participant must not be able to influence. This section formalizes the two-sided information boundary, the gate-crossing test, the referee state machine and arbitration order, the penalty model and the scoring rules, and it closes by treating the boundary as something to be measured rather than asserted.

7.1. *The Two-Sided Boundary*

The benchmark partitions the run state into two disjoint views. Writing x_t for the full simulator state at step t, the *controller view* of participant i is

$$o_{i,t} = \Phi_i(x_t) = (\, t_i^{\mathrm{loc}},\ \mathcal{Z}_{i,t},\ \mathcal{A}_{i,t},\ \mathcal{M}_{i,t}\,), \tag{15}$$

where t_i^{loc} is time since this participant's release, $\mathcal{Z}_{i,t}$ the allow-listed onboard measurements, $\mathcal{A}_{i,t}$ the set of acoustic packets physically received at step t, and $\mathcal{M}_{i,t}$ the optional teammate inbox. The *evaluator view* is

$$e_t = \Psi(x_t) = (\, \{p_{i,t},\ \dot{p}_{i,t}\}_{i\in\mathcal{P}},\ \mathcal{G},\ \mathcal{O},\ \mathcal{B},\ \mathbf{v}_c\,), \tag{16}$$

comprising exact vehicle poses and velocities, the ordered gate geometry, the obstacle set, the world bounds and the true current field. The benchmark enforces

$$\Phi_i \perp \sigma\Big(\ell_t,\ G_{i,t}^{\mathrm{done}},\ \mathrm{status}_{i,t},\ P_{i,t}\Big) \quad \text{for all } i, t, \tag{17}$$

that is, the observation map is independent of the referee's expected-gate index ℓ_t, of official completed-gate counts, of participant status and of accrued penalties. In the implementation this is a structural property rather than a convention: the observation builder's signature admits the track configuration, the arena, the adapter and the participant state, and does not admit a referee object, so referee-derived quantities are unreachable from the code path that constructs $o_{i,t}$.

Information flows in one direction only. The referee reads x_t and the controller's issued commands; the controller reads neither e_t nor any function of it. In particular, a controller is never told that a crossing was accepted, never told which gate is expected next, and never told that it has been penalized. Where the controller's own estimate of its progress disagrees with the referee's, the disagreement is logged and left standing (Section 7.7).

7.2. *Gate Frame and Crossing Test*

The referee reuses the right-handed gate frame $(\hat{n}, \hat{r}_g, \hat{s}_g)$ of (2), defined from the passage direction n_g with the vertical-normal fallback to the world $\hat{x}$ axis. For consecutive referee-side positions p_{t-1}, p_t, the signed distances to the gate plane are

$$d_{t-1} = \hat{n}^\top (p_{t-1} - c_g), \qquad d_t = \hat{n}^\top (p_t - c_g). \tag{18}$$

A candidate crossing requires a sign change in the *correct direction*,

$$d_{t-1} \le 0 \le d_t \ \wedge\ (p_t - p_{t-1})^\top \hat{n} > 0, \tag{19}$$

i.e. travel from the negative to the positive side along the normal. The intersection of the segment with the plane is

$$\lambda = \frac{-d_{t-1}}{d_t - d_{t-1}}, \quad q = p_{t-1} + \lambda\,(p_t - p_{t-1}), \quad \lambda \in [0, 1], \tag{20}$$

and the crossing is accepted only when q lies inside the aperture, shrunk by the configured safety margin m_g,

$$\left|\hat{r}_g^\top (q - c_g)\right| \le \tfrac{w_g}{2} - m_g, \qquad \left|\hat{s}_g^\top (q - c_g)\right| \le \tfrac{h_g}{2} - m_g. \tag{21}$$

Gates must be passed *in sequence*: the referee tests only the expected gate g_ℓ, advancing the index $\ell \leftarrow \ell + 1$ on a valid crossing and incrementing the lap when the sequence completes. Crossing the aperture of a *non-target* gate is a missed-gate event; a sign change in the wrong direction is logged but neither penalized nor counted as progress. Fig. 8 illustrates the test and Fig. 9 the principal automatic lifecycle.

Two design choices in this test are worth naming. Testing only the expected gate, rather than all gates, is what turns the course into an ordered task instead of a collection of independent hoops, and it is the reason the dominant learned-policy failure mode in Section 10.6 is a missed-gate termination rather than a low gate count. Shrinking the aperture by m_g before the containment test makes the pass criterion strictly conservative: a trajectory that clips the frame is not credited. The margin and the aperture dimensions were fixed before any experiment reported here and were never adjusted (requirement R4).

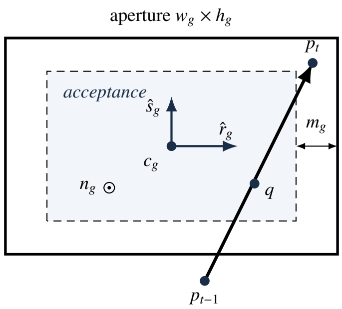

Figure 8: Gate-crossing geometry and the safety-margin-adjusted acceptance region.

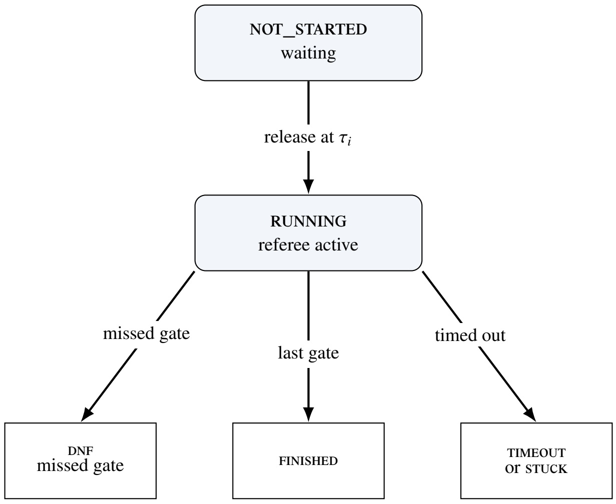

Figure 9: Simplified automatic participant lifecycle; manual-stop and controller-error terminals are omitted.

7.3. Referee State Machine and Arbitration

Each participant occupies one of the states NOT_STARTED, RUNNING, FINISHED, DNF, TIMEOUT, STUCK, DSQ, MANUAL_STOP or CONTROLLER_ERROR; all but the first two are terminal. A participant is released to RUNNING when the race clock reaches its start delay (Section 8). On each tick of a running participant, events are evaluated in a fixed priority order that defines the arbitration: (1) a controller error transitions to CONTROLLER_ERROR and returns; (2) out-of-bounds; (3) generic collision; (4) obstacle collision; (5) stuck accumulation; (6) duration timeout; (7) gate progression. Steps (2) to (5) accrue penalties without terminating; only (1), (6), a missed gate in (7) or an external gate-timeout produce a terminal state.

The order is not arbitrary. Placing controller errors first prevents a crashed participant from also accruing collision or stuck charges from its final frozen command. Placing gate progression last ensures that a tick in which the vehicle both crosses a gate and touches its frame is charged for the contact and credited for the crossing, rather than either event masking the other. To avoid double counting, an obstacle-collision tick suppresses the generic collision flag. Repeated collision, obstacle and out-of-bounds charges use independent cooldowns; inter-vehicle proximity uses its own team-level cooldown; stuck is latched once per episode; and terminal events are not cooldown-limited.

7.4. Penalties and Termination

Penalties accumulate into a per-participant total P_i; the applied defaults are listed in Table 8. The non-trivial conditions are formalized as follows. The instantaneous speed is $s_t = \|p_t - p_{t-1}\|/\Delta t$, and a stuck accumulator

Table 8: Default referee events, penalties and terminal conditions.

Event	Penalty	Terminal
Gate, bar or generic collision	5.0 s each	no
Obstacle collision	5.0 s each†	no
Out of bounds	10.0 s each	no
Stuck (soft)	15.0 s per episode	no
Missed gate	N/A	yes (DNF)
Timeout	N/A	yes
Gate-timeout stuck	N/A	yes
Inter-vehicle proximity	0 or 5.0 s per pair	no (team-level)

†Per-obstacle value if specified, else the collision default. Collision, obstacle, out-of-bounds and inter-vehicle proximity charges use independent configured cooldowns (default 1.0 s); stuck uses one latch per episode and terminal events have no cooldown.

integrates low-speed time,

$$T^{\text{stuck}} \leftarrow \begin{cases} T^{\text{stuck}} + \Delta t, & s_t < v_{\text{stuck}} \\ 0, & \text{otherwise}, \end{cases} \tag{22}$$

with a soft penalty charged once per continuous low-speed episode when $T^{\text{stuck}} \geq T^{\text{stuck}}_{\max}$ and $v_{\text{stuck}} = 0.02$ m/s. The configured $T^{\text{stuck}}_{\max}$ is 45 s on Horseshoe Bay and 65 s on Vertical Serpent and Mixed Endurance. An out-of-bounds event is $p_t \notin \mathcal{B}$. A duration timeout, disabled by default, fires when $t - t_{\text{green}} > T_{\max}$.

A missed gate sets DNF when `missed_gate_dnf` is enabled, which is the default, but carries no time penalty. The combination is deliberate: making a disqualifying error terminal but free of accumulated time ensures that losing the course is never cheaper than completing a slow but legal lap, which would otherwise be an exploitable scoring artefact.

7.5. Single-Vehicle Score and Ranking

For a finished participant the official time T_i^{official} is measured between the first and last gate (or green-to-finish, according to the timing mode) and the score is the penalized time

$$T_i^{\text{pen}} = T_i^{\text{official}} + P_i. \tag{23}$$

Ranking places finished participants before unfinished ones and is defined by the ascending lexicographic key

$$\kappa_i = \begin{cases} (0, T_i^{\text{pen}}, 0, 0, 0, \text{id}_i), & f_i = 1, \\ (1, -G_i^{\text{done}}, n_i^{\text{coll}}, P_i, \text{dist}_i, \text{id}_i), & f_i = 0, \end{cases} \tag{24}$$

where $f_i = 1$ denotes a finished vehicle, G_i^{done} the number of completed gates, n_i^{coll} the collision count and dist_i the distance to the next gate; the participant identifier is the deterministic final tie-break. Finished vehicles are therefore ordered by penalized time, and unfinished vehicles by progress, then by fewer collisions, fewer penalties and proximity to the next gate. This ordering preserves the information in partial runs instead of discarding them, which matters for the learned controllers of Section 9, most of whose official-circuit episodes are partial.

7.6. Event Logging

Every run emits a line-delimited event log and a machine-readable summary (requirement R5). Events include `gate_passed`, `wrong_direction`, `collision`, `obstacle_collision`, `out_of_bounds`, `stuck`, `race_finish` and `dnf`, each timestamped and attributed to a participant. For each participant, the summary records the status, the official and penalized times, the completed gates, the collision count, out-of-bounds events, missed gates and stuck events, together with the final ranking. In fleet mode it also carries the team summary of Section 8. Every table in Section 10 is an aggregation over these files.

Table 9: Controller-local course progression against independent referee progression, over the frozen 78-run matrix. A positive delay means the controller confirmed its own passage *after* the referee recorded the crossing. No value in this table is fed back into control.

	Gate advancements			Mismatches		Local delay (s)	
Circuit	referee	local	matched	false	missed	median	p95
Horseshoe Bay	1120	1120	1116	10	4	0.924	1.353
Vertical Serpent	159	160	159	1	0	0.627	1.155
Mixed Endurance	195	192	192	0	3	0.462	1.287
All	**1474**	**1472**	**1467**	**11**	**7**	**0.858**	**1.353**

A *false* local advancement is a local increment that precedes its same-ordinal referee crossing by more than one control step, or that has no corresponding referee crossing; a *missed* local advancement is a referee crossing with no same-ordinal local increment. Across all runs there were 0 event-consistency errors, 0 finish-order inversions over 61 pairwise comparisons, and 0 cases of a controller declaring itself finished before the referee agreed; the referee recorded the finish before the local tracker in 103 cases. The mean delay is 0.722 s with a sample standard deviation of 2.83 s; the spread is dominated by a single false advancement at −90.8 s, which is retained rather than filtered.

7.7. Measuring the Boundary

An information boundary that is only asserted is a design claim; one that is measured is evidence. Because the controller maintains its own course estimate (Section 6.3) and the referee maintains the official one, the two can be compared offline after every run, and the framework records that comparison automatically. We define a *false local advancement* as a local increment that precedes its same-ordinal referee crossing by more than one control step, or that has no corresponding referee crossing at all, and a *missed local advancement* as a referee crossing with no same-ordinal local increment in the run. The advancement delay is the controller-local advancement time minus the referee crossing time, so a positive delay means the controller learned of its own progress after the referee recorded it.

Table 9 reports this comparison over the frozen 78-run matrix. Across 1474 referee advancements the controller produced 1472 local advancements, of which 1467 matched the referee. The 11 false and 7 missed local advancements are 0.75% and 0.47% of the total. The controller consistently lags: the median delay is 0.858 s and the 95th percentile 1.353 s, which is the price of requiring camera, acoustic and DVL evidence to agree before advancing. No run recorded an event-consistency error, no finish order was inverted across 61 pairwise comparisons, and in no case did a controller declare itself finished before the referee agreed.

Two conclusions follow, and they are the reason we regard this table as central rather than incidental. First, the boundary is real: the controller's estimate is a genuinely separate quantity that is sometimes wrong, in both directions, and nothing in the framework repairs it. Second, the boundary is not *prohibitive*: an onboard-only controller can track an ordered course closely enough that its own progress estimate is correct in more than 99% of advancements, so requiring onboard-only operation does not make the task unmeasurable. The single negative delay of −90.8 s in the raw distribution is a false local advancement in which the controller advanced its expected beacon long before the referee accepted the corresponding crossing; it is retained in the audit rather than filtered, which is why the delay standard deviation (2.83 s) far exceeds its interquartile spread.

8. Fleet Execution, Coordination and Team-Level Evaluation

Fleet mode runs several vehicles as a single cooperative team through the same circuit, so the benchmark measures how quickly and how safely the team completes the course rather than pitting the vehicles against each other (requirement R7). The benchmark models one team: all vehicles share a single simulated world, each controller keeps an independent local course estimate, and the referee keeps one scoring state per vehicle. Enabling fleet mode does not alter single-vehicle behaviour (R6), so a fleet result and a single-vehicle result remain comparable.

8.1. Staggered Release and Spawn Geometry

Each participant i is assigned a release delay $\tau_i = (i-1)\,\Delta\tau$ for a start gap $\Delta\tau$, so vehicles enter the course at $0, \Delta\tau, 2\Delta\tau, \ldots$ A waiting vehicle receives a zero command, its controller `step` is *not* called, and neither stuck nor gate-timeout timers accumulate. At release the referee logs a `participant_released` event and resets the vehicle's progress timers, so a delayed start is never misclassified as stuck or timed out. Spawn poses are laterally separated about the base spawn to reduce launch interference: using zero-based $k = 0, \ldots, N-1$, the k-th vehicle is offset by

$$o_k = (-1)^k \left\lceil \frac{k}{2} \right\rceil s \tag{25}$$

along the right axis $(-\sin\psi_0, \cos\psi_0)$ of the base yaw ψ_0, giving the alternating pattern $0, -s, +s, -2s, +2s, \ldots$ for spacing s; a candidate that would fall outside the world bounds $\mathcal{B}$ is scaled inward until admissible.

8.2. *Inter-Vehicle Proximity Diagnostics*

Inter-vehicle proximity events are detected on the referee side and are never exposed to controllers, which is a direct consequence of (17): a vehicle that could read the referee's proximity flag would be receiving privileged information about a teammate's exact position. For two running vehicles a, b the detector applies a separable cylinder test

$$\sqrt{(x_a - x_b)^2 + (y_a - y_b)^2} \le r_{xy} \ \wedge \ |z_a - z_b| \le r_z, \tag{26}$$

with hysteresis — a pair is released only when the horizontal distance exceeds a release threshold r_{rel} or the vertical distance exceeds r_z — and a refractory cooldown to debounce sustained proximity. A vehicle that wants to avoid its teammates must therefore infer their presence from its own sensing and from messages they choose to send.

8.3. *Inter-Vehicle Acoustic Communication*

Because the fleet is a single cooperative team, Marine Race Arena provides an optional channel so that vehicles can share information while running the course. It is off by default and changes nothing for single-vehicle runs (R6). A controller transmits by returning a small `message` payload alongside its command and receives an `inbox` of teammates' messages in its observation; what a message contains and how it is used are entirely the participant's responsibility, so the framework provides only the transport.

To stay faithful to the medium rather than model a perfect radio link, the channel reproduces the constraints of the underwater acoustic channel discussed in Section 2.4. A message broadcast by vehicle i reaches a teammate j only if the two are within a maximum range $R_{\max}$, after a range-dependent delay

$$\tau_{ij} = \frac{d_{ij}}{c} + \tau_{\text{proc}}, \tag{27}$$

where d_{ij} is the inter-vehicle distance, c the speed of sound and τ_{proc} a fixed processing delay. Each delivery is dropped with probability p_{loss}. Payloads are capped to a small size to model the low bandwidth, and a vehicle may not transmit faster than a configured per-sender interval. Packet loss is drawn from a seeded generator, so a run remains reproducible.

We are explicit about the status of this component: it is an acoustic-inspired transport, not a characterized channel model. It provides the substrate for the coordination policy below and is exercised as part of that validation, but its packet loss, maximum range, latency and freshness window are not independently swept, and no claim about acoustic-communication performance is made in this paper.

8.4. *Distributed Leader–Follower Coordination*

Motivation. A staggered team of *identical* controllers stays separated in time, because each vehicle takes roughly as long as the one ahead. A *heterogeneous* team does not: a faster follower released behind a slower leader closes the gap, and the two then converge on the same aperture centre at the same gate, because both are servoing to the same visual target. The resulting contacts are a property of the convoy, not of either controller in isolation, and they are severe — Section 10.5 reports a mean of 127.3 gate and world collisions per run for an uncoordinated three-vehicle team at an 8 s gap.

Module design. We provide a coordination controller that maintains a spaced convoy using only legal information. It *wraps* rather than replaces a gate-passing controller that exposes a `LocalCourseTracker`, which keeps coordination and gate navigation separate concerns; both official camera-assisted controllers satisfy this interface. Each vehicle is assigned a predecessor from the static release order carried in its legal mission information; the frontmost vehicle has no predecessor and races freely. Within the channel's transmit-rate limit, every vehicle periodically broadcasts exactly three fields — `local_beacon_index`, `local_lap` and `local_status` — read from its own base controller's tracker. The receiver treats these as estimates, retains the most recently received predecessor report and ignores it after a 2.5 s freshness window.

Let $\hat{g}_i$ denote cumulative locally estimated progress reconstructed from the reported lap and beacon index. A follower yields while its predecessor is fewer than Δg locally estimated gates ahead,

$$\text{yield}_j \iff \hat{g}_{\text{pred}(j)} - \hat{g}_j < \Delta g, \tag{28}$$

and otherwise runs its wrapped base controller unchanged. While yielding, the wrapper still updates the base tracker from onboard observations but commands zero motion. A predecessor locally reported as finished, a missing report or a report older than the freshness window all stop blocking. With the channel disabled no report is delivered and the wrapper reduces exactly to the uncoordinated base behaviour.

Technical advantage and its cost. Every input to (28) is legal: the vehicle's own tracker, a static predecessor assignment and a controller-authored message that itself contains only the sender's own estimate. Referee progress can disagree with either local estimate but never changes a coordination decision. The cost is that a deliberate wait is indistinguishable, to the referee, from being stuck: yielding can trigger the independent stuck penalty of (22), and Section 10.5 shows this happening for the more conservative two-gate margin. We regard that as correct behaviour for a benchmark — a coordination policy that buys safety by standing still should pay for the time it spends standing still — and it is one reason the recommended default margin is one gate rather than two.

8.5. Team-Level Score

Only the `team_summary` is the official fleet result; per-vehicle rows are diagnostics. For a team $\mathcal{P}$ of N vehicles with G_{exp} expected gates each,

$$G_{\text{team}}^{\text{exp}} = N\,G_{\text{exp}}, \qquad G_{\text{team}}^{\text{done}} = \sum_{i\in\mathcal{P}} G_i^{\text{done}}. \tag{29}$$

The team finishes only if every vehicle finishes,

$$\text{finished}_{\text{team}} = \bigwedge_{i\in\mathcal{P}} \text{finished}_i, \tag{30}$$

in which case the team elapsed time spans the first release to the last finish,

$$T_{\text{team}} = \max_i T_i^{\text{finish}} - \min_i T_i^{\text{release}}, \tag{31}$$

and the penalized team score aggregates every per-vehicle penalty together with the team-level inter-vehicle term,

$$T_{\text{team}}^{\text{pen}} = T_{\text{team}} + \sum_{i\in\mathcal{P}} P_i + P_{\text{ivc}}, \tag{32}$$

where P_i is the per-vehicle penalty total of (23) and P_{ivc} counts each inter-vehicle proximity event once per pair rather than once per vehicle. The conjunction in (30) is what makes the score a *team* score: a fast leader cannot compensate for a follower that never finishes, so a coordination policy is rewarded for bringing the whole convoy home. Fig. 10 illustrates the aggregation.

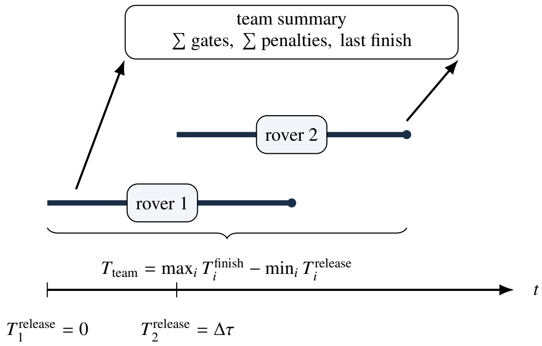

Figure 10: Team-level scoring for a staggered two-rover fleet.

9. Learning-Based Extension and Training Pipeline

A benchmark whose contract only rule-based controllers can satisfy would be of limited use. This section describes how a learned policy is integrated into Marine Race Arena without relaxing any part of the observation or referee contract, and documents the observation encoding, reward, curriculum, training pipeline, checkpoint-selection procedure and evaluation protocol that produced the policies evaluated in Section 10.6.

We state the scope of this section precisely. Its primary contribution is methodological: a learning integration that is, by construction, judged by the same referee and restricted to the same observation as every other controller. The learned policies we report are frozen artifacts with recorded hashes, and one of them failed a readiness gate that was registered before it was evaluated. We report that outcome as part of the result rather than around it, because the purpose of the benchmark is to make such outcomes visible.

9.1. Integration Through the Existing Contract

A learned controller enters the framework through the same three-method interface as any other participant (Section 6). A thin wrapper encodes the official observation into the policy's input vector, calls the policy, and returns the resulting four-channel command of (4), which is then clamped and depth-limited exactly as a rule controller's command would be. No component of the runner, the adapter or the referee is aware that a neural network is in the loop. In particular, the policy receives no gate coordinates, no global pose, no referee progress and no reward signal at evaluation time; the reward defined below exists only during training, where it may legitimately inspect privileged state because it is not part of the observation.

9.2. Observation Encoding

Motivation. A policy trained on a fixed-length course would learn the course. The encoding is therefore deliberately *sequence-length independent*: it describes the vehicle's relationship to its *current* target and its recent local dynamics, and contains no gate index, no mission length, no tracker phase and no future geometry. A policy that performs well under this encoding has learned a local transition skill that could in principle be chained arbitrarily, which makes chaining failures diagnosable rather than confounded with memorization.

Encoding. The contract `onboard_local_transition_v1` maps the official observation to 35 bounded features, grouped as follows:

- **Acoustic (7):** beacon presence, bearing as a sine/cosine pair, normalized elevation, normalized range, signal strength and normalized packet age.
- **Visual (5):** detection presence and the normalized centre (c_x, c_y), area fraction and confidence of the associated target from Section 5.2.
- **Depth (4):** normalized depth, presence, normalized depth error and reference presence.

- **Inertial and Doppler (6):** body-frame DVL surge, sway and heave with a presence flag; IMU yaw rate with a presence flag.
- **Previous action (4):** the four channels of the command issued at the previous step.
- **Local derivatives (9):** visual centre rates with a validity flag; beacon bearing, elevation and range rates with a validity flag; a recently-changed-target flag and the normalized time since the last target change.

Every feature is bounded in $[0, 1]$ or $[-1, 1]$ and every optional quantity carries an explicit presence flag, so a missing measurement is represented as absent rather than imputed. The presence flags matter given the perception audit of Section 5.4: a detection is unavailable in roughly one frame in ten, and the policy must be able to represent that state rather than receive a stale or fabricated value.

9.3. Action Space and Reward

The action space is the contract of (4): four continuous channels in $[-1, 1]$, identical to the rule controllers'.

The reward is shaped for reliability rather than speed and is designed so that oscillation cannot manufacture progress. All dense terms are *bounded signed deltas* of a monotone quantity, so moving toward the target and then back yields zero net reward. The terms are: bearing, elevation and range improvement toward the expected beacon; visual centring and apparent-area improvement; a gate crossing bonus; post-crossing alignment and range improvement toward the *next* target within a bounded window; and a completion bonus. Penalties cover collisions, out-of-bounds, wrong-direction crossings, missed gates, returns to a previously passed gate, acquisition timeouts, post-crossing orbiting and moving away from the target.

Two details of the collision term are worth recording because they were necessary in practice. Collision cost is charged per *entry* into contact rather than per contact frame, with a small bounded trickle for sustained contact and separate episode caps for entries and for contact frames. Without this distinction, a policy that lodges against a gate frame accumulates unbounded penalty from a single physical event, which destabilizes the value function. Each reward component and the total are additionally clipped to fixed bounds. A reward audit confirming these properties is included in the released artifacts.

The reward exposes two regimes. In the reliability regime the time, action-change, jerk and energy costs are zero or near-zero, so the policy is free to be slow. An efficiency regime, unlocked only after reliability criteria are met, activates time and smoothness costs. This staging follows directly from the benchmark's scoring rule: since a missed gate is terminal while a slow lap merely scores poorly (Section 7), optimizing speed before reliability optimizes the wrong objective.

9.4. Training Pipeline and Curriculum

Algorithm and architecture. Policies are trained with Proximal Policy Optimization [19] as implemented in Stable-Baselines3 2.7.1 [20] over a Gymnasium 1.0.0 environment that wraps the same arena, adapter and referee used for evaluation. The policy is a feed-forward Gaussian actor–critic with two hidden layers of 256 units in both the policy and value heads and tanh activations, consuming the 35-feature vector of Section 9.2 with a frame stack of one. Training used a linear learning-rate schedule from 3×10^{-5} to 5×10^{-6}, rollout length 2048, minibatch size 256, four epochs per update, $\gamma = 0.995$, $\lambda_{\text{GAE}} = 0.95$, clip range 0.10, entropy coefficient 10^{-3}, value coefficient 0.5, gradient-norm clipping at 0.5 and an initial action standard deviation of 0.12. A KL guard monitors each update against a target of 0.01, a high threshold of 0.02 and an absolute stop at 0.03, and at most one automatic adaptation is permitted per run so that a training curve remains interpretable.

Curriculum. The open problem is not crossing a gate but chaining crossings, so the curriculum controls the *mixture of episode lengths* rather than the geometry difficulty alone. Episodes are drawn from four buckets — focus (2 gates), short (3–5), medium (6–12) and long (13–24) — and four stages shift mass from the focus bucket toward long sequences: S_0 stabilization $(0.50, 0.35, 0.15, 0.00)$, S_1 chaining $(0.30, 0.35, 0.25, 0.10)$, S_2 balanced $(0.20, 0.30, 0.30, 0.20)$ and S_3 long-sequence $(0.10, 0.20, 0.35, 0.35)$. Short episodes are never removed, because they remain the highest-signal-per-second source of crossing and target-switch experience.

Promotion between stages is driven by measured generalization on an unseen validation seed band rather than by elapsed transitions: a run that has merely survived many steps has not earned harder work. Each stage carries explicit promotion thresholds — for example S_1 requires a universal-transition success rate of 0.87 and a short-sequence completion of 0.82, sustained over two consecutive qualifying evaluations — and S_3 is terminal. Course geometry is sampled from a procedural family; the three official circuits are never used for training.

Retention. Where a run is warm-started from a behaviour-cloning reference, an offline retention regularizer applies one bounded auxiliary update after each configured PPO update, computed only on recorded observation–action pairs and on a KL term against the initial policy, with a weight that decays to zero over training. The regularizer never constructs or queries an expert controller inside the live rollout environment, which keeps the on-policy data free of expert intervention.

9.5. Checkpoint Selection and the Pre-Registered Readiness Gate

Selecting a checkpoint by its best training-time metric invites selection on noise, so selection is separated from training by a pre-registered gate. Concretely, the training run is frozen first; the resulting checkpoint is written with an exclusive-create freeze record carrying its SHA-256, the git commit at freeze time and the total transition count, and marked read-only on disk. The readiness thresholds are registered before the evaluation runs, and the threshold object is itself hashed inside the freeze record, so a later edit to the defaults would be detectable.

The gate evaluates 19 criteria on a validation seed band disjoint from training: a 500-case universal-transition benchmark, per-length full-sequence completion at 3, 5, 8, 12, 17 and 22 gates with 20 cases each, a six-rung difficulty ladder, and safety and protocol criteria. The decisive distinction it enforces is between *survivor-conditioned* and *unconditional* rates: reporting the success of a transition given that the policy reached that point flatters a policy that rarely gets there, so the gate also records the unconditional probability of reaching gate k from episode start.

Section 10.6 reports the outcome. We note here only that the gate returned FAIL for the frozen Generation-1 policy on four of its nineteen criteria, that no manual override was requested or applied, and that the policy was nonetheless carried into the final holdout evaluation as registered, so that the relationship between a failed procedural gate and downstream circuit performance could be observed rather than assumed.

9.6. Evaluation Protocol for Learned Controllers

Learned controllers are evaluated exactly like any other participant, with three additional protocol constraints.

First, *seed separation*. Seed bands are registered per role — training, validation, test and final holdout — in a seed registry committed to the repository, and the evaluation records both requested and completed seeds. The matched benchmark of Section 10.6 runs every controller on the identical seed set and the identical generated geometries, so all comparisons are paired.

Second, *policy purity*. Each evaluation records whether every action was produced by the policy. A controller that blends a learned output with a rule controller's output is marked as not policy-only and reported separately; the hybrid controller in Section 10.6 is labelled accordingly and is excluded from any claim about learned control.

Third, *artifact provenance*. Every evaluation manifest records the git commit, the model path and SHA-256, the track file and its SHA-256, the requested adapter and the adapter actually used, whether the engine-free fallback was permitted, the current profile and the measured current vector, the control timestep, and the observation and action contract versions. A run whose `adapter_actual` is not `holoocean` or whose `fallback_allowed` is true does not enter any table in this paper.

Finally, a scoping constraint that applies to every learned-controller result on the three official circuits. Those circuits were opened by an exploratory holdout evaluation whose access ledger records all sixty accesses. They are therefore *not* unseen holdouts for any subsequent policy, and we report learned-controller results on them as a retrospective diagnostic benchmark, never as a held-out generalization claim.

10. Experimental Evaluation

This evaluation does not seek an optimal underwater racing controller. It asks whether the benchmark, the observation contract, the referee and the team scoring behave as specified, and whether they expose differences between autonomy strategies that a coarser protocol would hide. We organize the experiments around five research questions rather than chronologically, and we report failures of the reference controllers and of the learned policies as results rather than as caveats.

RQ1 Can Marine Race Arena support fully onboard autonomous completion across heterogeneous underwater racing circuits?

RQ2 Does the benchmark expose meaningful differences between autonomous control strategies?

RQ3 How do environmental currents affect benchmark performance?

RQ4 Can the same evaluation framework support multi-vehicle and team-level racing?

RQ5 Can learning-based controllers be integrated and evaluated under the same observation and referee contract?

10.1. Protocol, Metrics and Provenance

Three independent bodies of evidence are reported, and they are never pooled.

The systematic benchmark matrix. 78 real-HoloOcean runs generated under a frozen source-tree fingerprint recorded in the artifact manifest. It covers both reference controllers on the three clean circuits and under the `medium` and `strong` Horseshoe Bay currents, homogeneous two-vehicle fleets, and matched three-vehicle coordination trials. Clean, current and fleet cases use five seeds; coordination uses three seeds under simultaneous and staggered release. Control runs at $\Delta t = 0.033$ s (30 Hz). This matrix answers RQ2, RQ3 and RQ4, and supplies the boundary measurement of Section 7.7.

The current-free completion demonstration. A focused nine-run demonstration that the reference controller completes each official circuit end to end with currents explicitly disabled and verified zero in the manifest, at $\Delta t = 0.1$ s (10 Hz). This answers RQ1.

The matched controller benchmark. 474 episodes in which six controllers — one rule-based, one hybrid, three PPO checkpoints and one behaviour-cloning policy — ran an identical suite of test groups, generated and official geometries, seeds, adapter and simulator configuration, at $\Delta t = 0.1$ s. This answers RQ5.

All performance trials use the reference HoloOcean adapter, a BlueROV2-class vehicle, official onboard-only observations, no current compensation and no obstacles, with the standard gate apertures and referee margin unchanged throughout (R4). Separate setup checks confirm that both named current profiles and seeded static obstacles instantiate on every circuit; these are construction checks, not obstacle-avoidance trials, and no obstacle result is claimed.

The reported metrics are completion, completed gates, official and penalized time, collisions, out-of-bounds events, wrong-direction crossings and stuck events. Fleet and coordination trials add inter-vehicle proximity events and team times. Time statistics use finished runs only and are undefined where nothing finished; gate and event means use all seeds. Dispersions are sample standard deviations (ddof = 1). Proportions carry Wilson 95% intervals where sample sizes are small enough for the normal approximation to mislead. Paired comparisons use two-sided sign tests on matched seeds.

Every run records the experiment-generation commit, the source-tree fingerprint or model SHA-256, the HoloOcean version, the track file and its SHA-256, the requested and actual adapter, the fallback-disabled status, the current profile and measured current vector, the control timestep and the seed. A run whose actual adapter was not HoloOcean, or which permitted the engine-free fallback, does not appear in any table in this paper.

10.2. RQ1: Onboard-Only Completion Across Heterogeneous Circuits

The first question a racing benchmark must answer about itself is whether its task is solvable at all under its own restrictions. If no controller can complete a circuit using only onboard sensing, the benchmark measures the difficulty of an impossible problem rather than the quality of autonomy.

Table 10 answers this affirmatively and without qualification for the current-free setting. The Center-then-Commit controller completed all three official circuits in every one of nine runs: 3/3 on Horseshoe Bay (12 gates), 3/3 on Vertical Serpent (17 gates) and 3/3 on Mixed Endurance (22 gates), for 51 of 51 gates in total. Across those nine runs the referee recorded zero gate, bar or world collisions, zero out-of-bounds events and zero wrong-direction crossings, and penalized time equalled official time in every run because no penalty was ever charged. Every run manifest independently confirms the conditions under which this was achieved: the real HoloOcean adapter was used, the

Table 10: Current-free autonomous completion of the three official circuits by the Center-then-Commit reference controller. Real HoloOcean adapter, engine-free fallback disabled, currents verified zero in the manifest, no privileged controller input, $\Delta t = 0.1$ s, seeds 1800–1802.

Circuit	Gates	Runs	Completed	Mean gates	Official time (s)	Coll.	OOB / WD
Horseshoe Bay	12	3	3/3	12.0/12	236.0 ± 6.9	0	0 / 0
Vertical Serpent	17	3	3/3	17.0/17	308.6 ± 6.5	0	0 / 0
Mixed Endurance	22	3	3/3	22.0/22	480.4 ± 15.7	0	0 / 0
Total	**51**	**9**	**9/9**	**51.0/51**	—	**0**	**0 / 0**

Times are mean ± sample standard deviation (ddof= 1) over the three runs; penalized time equals official time in every run because no penalty was charged. *Coll.*: gate, bar and world collision events. *OOB / WD*: out-of-bounds events and wrong-direction crossings. Every run manifest records `adapter_actual` = `holoocean`, `fallback_allowed` = false, `current_profile` = `none` and `currents_actual` = [0, 0, 0], together with the track SHA-256 and the source commit. Mixed Endurance is a `current_gate` task whose validation requires a current, so a current-free run additionally overrides the validation mode to `clean_gate`; the track geometry, gate order, laps and referee configuration are unchanged. These runs use a 10 Hz control rate and are therefore not directly comparable in time with Table 11, which uses 30 Hz.

Table 11: Systematic clean-track evaluation of the two reference controllers over five seeds per case, from the frozen 78-run HoloOcean matrix ($\Delta t = 0.033$ s, 30 Hz control). Times use finished runs only; gate and event means use all five seeds.

Track	Controller	Seeds	Finished	Gates ↑	Official time (s) ↓	Collisions ↓
Horseshoe Bay	Continuous Servo	5	5/5	12.0/12	194.5 ± 4.8	0.0
	Center-then-Commit	5	5/5	12.0/12	197.7 ± 7.1	0.0
Vertical Serpent	Continuous Servo	5	**5/5**	**17.0/17**	236.9 ± 6.4	0.0
	Center-then-Commit	5	4/5	14.8/17	241.1 ± 1.8	0.0
Mixed Endurance	Continuous Servo	5	2/5	17.0/22	394.7 ± 3.9	0.0
	Center-then-Commit	5	**5/5**	**22.0/22**	410.7 ± 8.8	0.0

Times are mean ± sample standard deviation (ddof= 1) over finished runs; gates and collisions are five-seed means, and collisions combine gate and world events. No clean run recorded an out-of-bounds or stuck event, and penalized time equals official time throughout. Bold marks the better controller where the two differ in completion. Regenerated from the raw per-run summaries by `article/regenerate_tables.py`.

engine-free fallback was disabled, the requested and measured current vector was [0, 0, 0], and the controller received no privileged input.

Completion time scales with course length roughly as expected from the track geometry: 236.0 ± 6.9 s over 93.8 m, 308.6 ± 6.5 s over 118.3 m and 480.4 ± 15.7 s over 206.3 m, i.e. approximately 0.40, 0.38 and 0.43 m/s of course progress. The consistency of that rate across three structurally different circuits — planar, vertically serpentine and long mixed — indicates that the limiting factor is the controller's gate-by-gate acquire–align–commit cycle rather than any circuit-specific difficulty.

We are deliberate about the scope of this result and about its relationship to the systematic matrix. This is a *focused completion demonstration*: three seeds per circuit, one controller, no disturbance, at a 10 Hz control rate. It establishes that the task is achievable end to end under the full observation contract, and it does so with a per-run manifest that makes the claim auditable. It is not a performance characterization, and it does not supersede the broader evaluation of Sections 10.3–10.5, which uses five seeds, both controllers, three disturbance conditions and a 30 Hz control rate. The two are complementary and are reported separately for that reason; in particular, the times in Table 10 are systematically longer than those in Table 11 because the control rate differs by a factor of three, and the two must not be compared.

10.3. RQ2: Do Control Strategies Separate?

A benchmark that ranks every reasonable controller identically is not measuring anything. The two reference controllers of Section 6 are an unusually clean test of this, because they differ in exactly one respect — what happens during COMMIT and VERIFY_EXIT — while sharing observations, guidance, tracker and confirmation logic.

Table 11 and Fig. 11 show that the answer depends on the circuit, and that neither controller dominates. On the short, largely planar Horseshoe Bay circuit the two are effectively tied: both complete all 12 gates in all five seeds with no collision, out-of-bounds or stuck event, and the staged controller is 3.2 s slower (197.7 ± 7.1 s versus 194.5 ± 4.8 s),

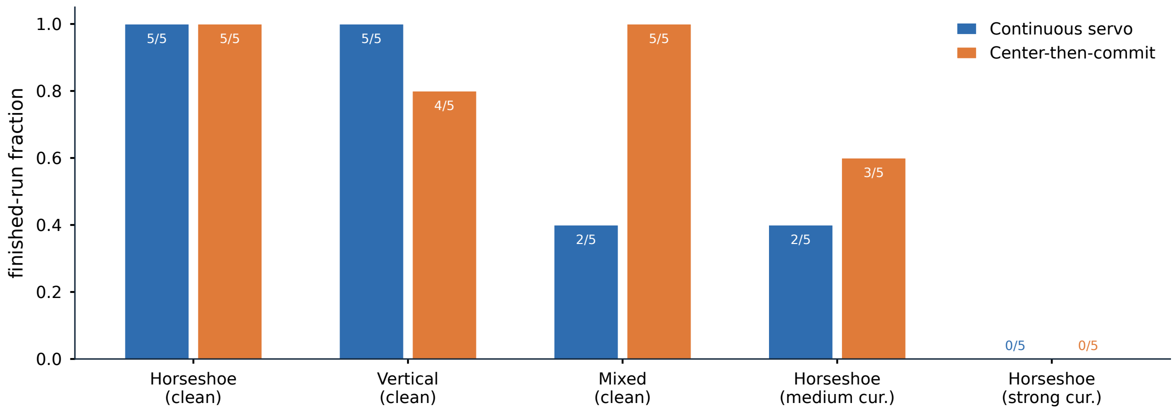

Figure 11: Finished-run fraction of the two reference controllers over five seeds per condition, from the frozen 78-run matrix; each label reports finished runs out of five. Neither controller dominates: they tie on the short planar circuit, Continuous Servo leads on Vertical Serpent, and Center-then-Commit leads on Mixed Endurance and under the medium current.

a 1.7% penalty with no measured safety benefit. Constant re-centring is cheap here, and paying nothing for it is the right trade.

The controllers separate on the harder circuits, and they separate in opposite directions. On Vertical Serpent, Continuous Servo is more reliable, finishing 5/5 against 4/5 and completing 17.0 against 14.8 mean gates. On Mixed Endurance, which is more than twice as long, the ordering reverses far more sharply: Center-then-Commit finishes all five seeds and completes every gate (22.0/22), while Continuous Servo finishes only 2/5 and averages 17.0/22.

The mechanism is consistent with the design argument of Section 6.5. Continuous visual correction is harmless when a passage is short and the approach is well conditioned, and it becomes a liability when partial near-field contours make the image centroid move quickly at exactly the moment the aperture margin is smallest. A long course compounds this: each passage is an independent opportunity for a late correction to fail, so a small per-gate risk becomes a large per-course risk. That Continuous Servo nonetheless wins on Vertical Serpent shows the trade is not one-directional — repeated vertical transitions appear to benefit from continuous re-centring more than they suffer from late corrections — which is precisely the kind of condition-dependent result a single-circuit benchmark would have concealed. The comparison is limited to the three official circuits, two current profiles and the evaluated seeds.

10.4. RQ3: Behaviour Under Environmental Currents

Performance degrades sharply and non-uniformly as current strength increases (Table 12). Under the `medium` Horseshoe Bay profile, Continuous Servo finishes 2/5 runs, averages 8.4/12 completed gates and 43.4 gate and world collisions per run, with finished-only official and penalized times of 337.9 ± 19.2 s and 692.9 ± 75.7 s. Center-then-Commit finishes 3/5, averages 8.8/12 gates and 25.2 collisions, with finished-only times of 217.3 ± 1.5 s and 223.9 ± 1.8 s. Neither controller finishes a single `strong` run: Continuous Servo averages 3.0/12 gates with no gate or world collision, and Center-then-Commit 3.2/12 gates with 0.8.

Two features of these numbers require comment, because both are easy to misread.

First, the collision counts are *non-monotonic* in current strength: the medium profile records many collisions (43.4 and 25.2) while the strong profile records almost none (0.0 and 0.8). This is not evidence of better control under the stronger disturbance. Under `strong` the vehicles complete only 3.0–3.2 of 12 gates and stall early, so they rarely reach the gate-dense region of the circuit where contacts accumulate. Fewer collisions here reflect less progress, not more skill, and a benchmark that reported collision counts without completion would invert the ranking.

Second, the `strong` profile is best read as a saturation regime rather than a graded difficulty. Its constant set-flow component alone is $[0.75, 1.05, 0]$ m/s, a magnitude of 1.29 m/s, about 2.7× the mean course speed the same vehicle sustains on clean Horseshoe Bay (93.8 m in 194.5 s, i.e. ≈ 0.48 m/s). No controller restricted to this vehicle's actuation can make sustained forward progress against it, so the condition functions as a stress case that confirms the disturbance is physically coupled rather than as an informative comparison point. The `medium` profile (0.65 m/s

Table 12: Robustness to environmental currents on Horseshoe Bay, five seeds per case, from the frozen 78-run matrix ($\Delta t = 0.033$ s). The clean row is repeated from Table 11 as the zero-disturbance reference.

Profile	Controller	Seeds	Finished	Gates ↑	Official (s)	Penalized (s)	Collisions ↓
None	Continuous Servo	5	5/5	12.0/12	194.5 ± 4.8	194.5 ± 4.8	0.0
	Center-then-Commit	5	5/5	12.0/12	197.7 ± 7.1	197.7 ± 7.1	0.0
`medium`	Continuous Servo	5	2/5	8.4/12	337.9 ± 19.2	692.9 ± 75.7	43.4
	Center-then-Commit	5	**3/5**	**8.8/12**	217.3 ± 1.5	223.9 ± 1.8	**25.2**
`strong`	Continuous Servo	5	0/5	3.0/12	—	—	0.0
	Center-then-Commit	5	0/5	3.2/12	—	—	0.8

Times are mean ± sample standard deviation over finished runs only and are undefined where no run finished. Gates and collisions are five-seed means, and collisions combine gate and world events. The `medium` profile has a constant set-flow component of 0.65 m/s and `strong` of 1.29 m/s, against a mean clean course speed of ≈0.48 m/s for the same vehicle on this circuit. The near-zero collision counts under `strong` reflect less progress, not better control: both controllers stall after about three gates and rarely reach the gate-dense part of the course. Run metadata confirms physical current coupling in every run.

constant component) is the informative regime, and there the staged controller is clearly the more robust: more completions, roughly half the collisions, and finished runs 120.6 s faster.

The headline finding for benchmark design is the gap between Table 11 and Table 12. Both controllers score a perfect 5/5 with zero events on clean Horseshoe Bay, and drop to 2/5 and 3/5 with tens of collisions on the same circuit under a moderate current. Clean-track success is therefore not a proxy for robustness, and any evaluation protocol that reports only clean completion will rank controllers on a dimension that disappears the moment the water moves. The quantitative current sweep is limited to Horseshoe Bay (Section 12).

10.5. RQ4: Multi-Vehicle and Team-Level Evaluation

The upper block of Table 13 validates that team execution works. With either controller, a homogeneous two-vehicle fleet on clean Horseshoe Bay at a 90 s release gap finishes 24/24 team gates in all five seeds, with no gate or world collision, no inter-vehicle proximity event, no out-of-bounds and no stuck event. Mean team elapsed and penalized times coincide at 303.1 ± 3.2 s for Continuous Servo and 308.0 ± 5.8 s for Center-then-Commit. This is deliberately a low-contact case: it exercises multi-vehicle spawning, staggered release, independent per-vehicle referee state, ranking and team aggregation without the vehicles interacting, and it confirms requirement R6, since the per-vehicle behaviour matches the single-vehicle case.

The interacting case is the interesting one. A heterogeneous three-vehicle convoy — a Continuous Servo leader followed by two Center-then-Commit vehicles — converges on the same apertures, and the consequences are severe. At an 8 s gap the uncoordinated convoy finishes only 2/3 runs, averages 34.3/36 team gates, 127.3 gate and world collisions and 2.0 inter-vehicle proximity events per run. At simultaneous release it still finishes 3/3, but averages 9.3 collisions and 2.3 proximity events.

The distributed leader–follower policy of (28) removes this entirely. At both gaps, LF(1) finishes 3/3 with 36.0/36 team gates and zero gate, world, proximity, out-of-bounds and stuck events, at 266.0 ± 8.3 s (gap 8 s) and 262.3 ± 7.7 s (gap 0 s). The more conservative LF(2) also eliminates contact but is consistently slower (285.7±3.8 s and 288.3±1.3 s) and, at simultaneous release, its longer hold trips the independent stuck timer once per run, raising its penalized team time to 303.3 ± 1.3 s. A larger nominal gate separation therefore bought no additional safety in these conditions while costing both time and a penalty, which is why LF(1) is the recommended default and LF(2) is retained only as a conservative comparison.

Two properties of this result matter beyond the numbers. The coordination policy consumes only the vehicle's own tracker state, a static predecessor assignment and teammate-authored messages delivered over a channel with range-dependent latency and loss; it never reads referee progress or a teammate's true position, so the improvement is achieved inside the same information boundary that constrains everything else. And the cost of coordination is visible in the score rather than hidden: LF(2)'s stuck penalty is charged by the same independent referee that charges a collision, so a policy that buys safety by waiting pays for the waiting. This evidence is limited to one clean circuit, three vehicles, two release gaps and three seeds.

Table 13: Multi-vehicle evaluation on clean Horseshoe Bay, from the frozen 78-run matrix ($\Delta t = 0.033$ s). The upper block validates team execution with a homogeneous two-vehicle fleet at a 90 s release gap (five seeds); the lower block studies a heterogeneous three-vehicle convoy under two release gaps (three seeds per condition).

Setting	Policy	Seeds	Team finish	Team gates ↑	Team time (s) ↓	GW / IV / S ↓
Two vehicles, homogeneous, gap 90 s						
	Continuous Servo	5	5/5	24.0/24	303.1 ± 3.2	0.0 / 0.0 / 0.0
	Center-then-Commit	5	5/5	24.0/24	308.0 ± 5.8	0.0 / 0.0 / 0.0
Three vehicles, heterogeneous convoy, gap 0 s						
	No coordination	3	3/3	36.0/36	219.7 ± 2.5	9.3 / 2.3 / 0.0
	LF(1)	3	3/3	36.0/36	262.3 ± 7.7	**0.0 / 0.0 / 0.0**
	LF(2)	3	3/3	36.0/36	288.3 ± 1.3	0.0 / 0.0 / 1.0
Three vehicles, heterogeneous convoy, gap 8 s						
	No coordination	3	2/3	34.3/36	254.0 ± 24.5	127.3 / 2.0 / 0.0
	LF(1)	3	**3/3**	**36.0/36**	266.0 ± 8.3	**0.0 / 0.0 / 0.0**
	LF(2)	3	3/3	36.0/36	285.7 ± 3.8	0.0 / 0.0 / 0.0

Team times are raw team elapsed mean ± sample standard deviation over full-team finishes, computed by (31); gates and events are per-condition means. *GW / IV / S*: gate and world collisions, inter-vehicle proximity events and stuck events. No run recorded an out-of-bounds event. LF(Δg) is the leader–follower policy of (28) with a margin of Δg locally estimated gates. Penalized team time differs from raw team time only where events were charged: 266.4 ± 64.1 s and 564.0 ± 463.0 s for the uncoordinated convoy at gaps 0 and 8 s, and 303.3 ± 1.3 s for LF(2) at gap 0 s, whose longer hold trips the independent stuck timer of (22). In the three-vehicle condition the leader runs Continuous Servo and both followers run Center-then-Commit; no controller or course parameter was retuned between conditions.

10.6. RQ5: Learned Controllers Under the Same Contract

Table 14 and Fig. 12 report the 474-episode matched benchmark. The integration question is answered first and plainly: five controllers of three different kinds — rule-based, hybrid and learned — ran the identical suite through the identical interface, were scored by the same referee, and produced directly comparable, paired results. No part of the runner, adapter or referee was modified to accommodate a neural policy, and the evaluation manifests confirm that every action of the policy-only controllers was policy-generated.

The performance question has a more interesting answer than a single ranking would suggest. On the eight generated test groups of one to three gates, learned policies are competitive with the rule baseline: PPO 525k matches it at 100%, PPO 1000k reaches 96.9% and PPO 900k 90.6%, against the baseline's 100%. On the three full official circuits the picture inverts: the rule baseline completes 93.3% while the best PPO checkpoint reaches 40.0%, the others 26.7% and 20.0%, and the behaviour-cloning policy 0%.

Table 15 shows that "worse" is the wrong summary. Over the 79 episodes each controller shares with the rule baseline on identical seeds and geometries, all three PPO checkpoints are simultaneously *significantly less reliable* (sign-test $p \leq 0.001$ on discordant completion pairs) and *significantly faster* on the episodes they do complete (Δt between -6.77 and -7.05 s, $p \leq 0.001$). The learned policies are not degraded versions of the rule controller; they occupy a different point on the speed–reliability trade-off, and they reach it without any of the hand-designed staging that the rule controller uses.

The failure is sequential, not physical. The decisive diagnostic is *how* learned policies fail. Of the 32 PPO failures on official circuits, 29 (90.6%) are referee-terminal missed-gate events and only 3 are time limits; none is an out-of-bounds termination. The median failure occurs after completing 23.9% of the course. The policies are not crashing into gates or leaving the world; they are crossing a gate and then failing to acquire the next one, at which point the referee's ordered-sequence test (Section 7) terminates the run.

Fig. 13 makes the same point quantitatively for the frozen Generation-1 policy, whose readiness evaluation instrumented survival explicitly. The probability of crossing the first gate from episode start is 1.000 (120/120), and it falls to 0.733 (88/120) for the full gate-1 to gate-2 transition, 0.575 at gate 8 and 0.300 at gate 22. Survivor-conditioned transition rates conceal this: measured over all 500 transition cases the universal transition success rate is 0.820, which looks like a policy that usually chains correctly, whereas the unconditional survival curve shows that more than a quarter of episodes are lost at the very first transition. Reporting conditional rates alone would have made a policy that cannot chain look like a policy that chains well.

Table 14: Matched benchmark of six controllers over 474 episodes on identical seeds and identical geometries. Real HoloOcean adapter, currents disabled, $\Delta t = 0.1$ s. The upper block aggregates eight generated test groups of one to three gates (64 episodes per controller); the lower block aggregates the three official circuits (15 episodes per controller).

Controller	Kind	Ep.	Completion	95% CI	Safety ep.	Coll. ev.	OOB ev.
Generated geometries, 1–3 gates							
Center-then-Commit	rule	64	**100.0%**	[0.94, 1.00]	**0**	**0**	0
Hybrid	hybrid†	64	100.0%	[0.94, 1.00]	2	35	0
PPO 525k	learned	64	100.0%	[0.94, 1.00]	4	16	0
PPO 1000k	learned	64	96.9%	[0.89, 0.99]	13	156	0
PPO 900k	learned	64	90.6%	[0.81, 0.96]	6	14	0
BC-v3	learned	64	89.1%	[0.79, 0.95]	3	36	0
Official circuits							
Center-then-Commit	rule	15	**93.3%**	[0.70, 0.99]	**1**	**3**	**0**
Hybrid	hybrid†	15	86.7%	[0.62, 0.96]	2	31	0
PPO 900k	learned	15	40.0%	[0.20, 0.64]	8	345	0
PPO 1000k	learned	15	26.7%	[0.11, 0.52]	10	1516	33
PPO 525k	learned	15	20.0%	[0.07, 0.45]	13	1434	92
BC-v3	learned	15	0.0%	[0.00, 0.20]	10	1200	468

Confidence intervals are Wilson 95% intervals on the completion rate. *Safety ep.*: episodes recording at least one safety event. *Coll. ev.*: total collision events. †The hybrid controller blends a deterministic backbone with a learned visual servo and is therefore not policy-only; the benchmark reports it separately and it supports no claim about learned control. Every PPO checkpoint is identified in the released manifest by SHA-256. Because the three official circuits were opened by an earlier exploratory holdout whose access ledger records all sixty accesses, the lower block is a *retrospective diagnostic* benchmark and not a held-out generalization result.

A pre-registered negative result. Table 16 reports the readiness gate and the paired final holdout for that policy. The gate returned FAIL on 4 of its 19 registered criteria — universal transition success (0.820 against a bound of 0.900) and completion at 3, 5 and 8 gates — with no override requested or applied. Carried into the holdout as registered, the frozen policy completed 0 of 30 official-circuit trials while the paired rule controller completed 30 of 30, crossing 87 of 510 gates; every PPO episode crossed at least gate 1 and then terminated on a missed gate at the next attempted gate. The consistency between the procedural gate and the downstream outcome is itself informative: a pre-registered chaining criterion measured on generated geometry predicted circuit-level failure, which is what such a gate is for.

What this says about the benchmark. These results are reported as a retrospective diagnostic rather than a held-out generalization claim, because the three official circuits were opened by the exploratory holdout whose ledger records all sixty accesses (Section 9.6). With that scoping stated, the finding stands: under a fixed observation contract and an impartial referee, current PPO policies trained on procedural geometry learn a competent single-gate transition and fail to chain it over a long ordered sequence. The benchmark's ordered-gate termination rule is what makes this visible; a scoring rule that credited any gate crossing would have reported the same policies as broadly successful.

11. Discussion

The experiments of Section 10 are individually unsurprising and collectively point at something more general: for underwater autonomy, the difficulty is concentrated in places that a conventional control benchmark does not probe. This section develops that argument.

11.1. Why Racing Is a Useful Benchmark for Underwater Autonomy

Racing is often justified by speed, but speed is not what makes it valuable here. An ordered gate course imposes three requirements that a station-keeping, trajectory-tracking or docking task does not. It requires *sequential commitment*: the vehicle must decide which of several possible targets is the right one, and a wrong commitment is unrecoverable rather than merely costly. It requires *repeated re-acquisition*: after each passage the target changes discontinuously, and the cue that identified the old target now actively misleads, because the just-cleared beacon sits behind the vehicle while the next gate is visible ahead. And it requires *sustained composition*: a per-gate success

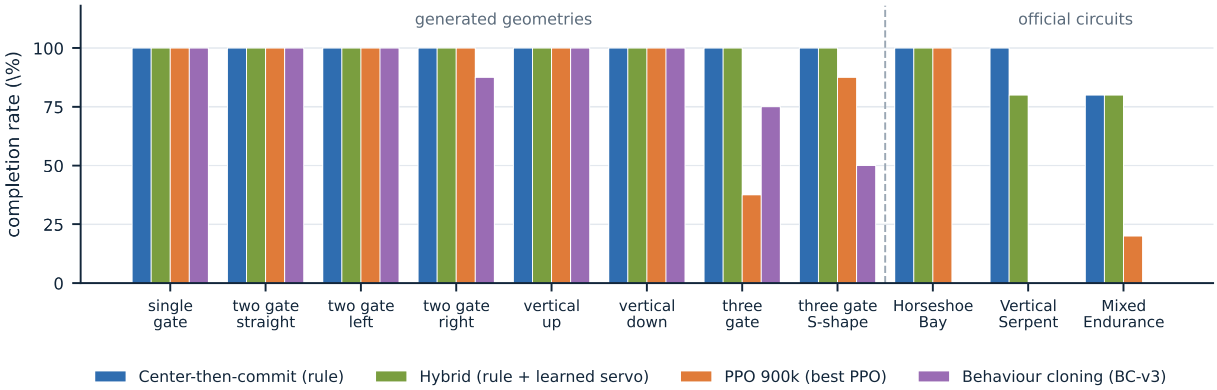

Figure 12: Completion rate per test group in the 474-episode matched benchmark, on identical seeds and geometries. Learned controllers are competitive on the generated one-to-three-gate geometries and collapse on the full official circuits, while the rule baseline is nearly flat across both regimes. The hybrid controller is not policy-only and is shown for reference.

Table 15: Paired per-seed comparison of each learned or hybrid controller against the Center-then-Commit rule baseline over the 79 episodes both ran on identical seeds and geometries. Negative Δt means the controller was faster than the rule baseline.

Controller vs. rule	Paired ep.	Only A fin.	Only rule fin.	Reliability p	Δt (s) [95% CI]	Time p
BC-v3	79	0	21	<0.001	+2.84 [+0.79, +4.89]	0.0002
Hybrid	79	1	2	1.000	+8.51 [+1.01, +16.02]	0.807
PPO 1000k	79	0	12	0.0005	**-6.77** [−10.91, −2.63]	0.0001
PPO 525k	79	0	11	0.0010	**-7.02** [−10.11, −3.92]	<0.001
PPO 900k	79	0	14	0.0001	**-7.05** [−11.59, −2.51]	<0.001

Only A fin. and *Only rule fin.* count the paired episodes in which exactly one of the two controllers finished. Reliability p is a two-sided sign test on those discordant pairs; time p is a two-sided sign test on the paired completion-time differences of episodes both controllers finished. Timing differences are computed within a test group and never across groups, so a completion time on one geometry is never weighed against a completion time on another. The pattern is consistent across all three PPO checkpoints: each is *significantly less reliable* and *significantly faster* than the rule baseline.

rate of 0.95 yields a 22-gate completion rate of 0.32, so a course of realistic length is a severe amplifier of small per-transition defects.

Our results show all three biting. The perception audit (Section 5.4) found multiple simultaneous gate candidates in 4 to 11 frames per 60-frame capture, so the commitment problem is real rather than hypothetical. The learned policies cross their first gate with probability 1.000 and lose more than a quarter of episodes at the very first transition, so re-acquisition, not crossing, is the binding skill. And the survival curve of Fig. 13 is the composition amplifier made visible. A benchmark built around a single manoeuvre would have measured none of these.

11.2. What Information Separation Buys

Enforcing the boundary of (17) has a cost — the controller must reconstruct course progress it could otherwise be handed — and it is worth being explicit about what the cost buys.

It buys comparability. Because the observation map is fixed and structurally independent of the referee, the 474-episode comparison of Section 10.6 attributes differences to the policy rather than to the information each policy was allowed. A hand-written controller and a neural network that see different inputs cannot be meaningfully ranked, and in the absence of an enforced contract they usually do.

It buys a measurable quantity where there would otherwise be an assumption. Because the controller maintains its own progress estimate and the referee maintains the official one, their disagreement is data (Table 9). The result — 1467 of 1474 advancements matched, with a median controller lag of 0.858 s, 11 false and 7 missed local advancements, and no finish-order inversion — is worth reading in both directions. The controller is nearly always right, which shows that onboard-only ordered navigation is tractable and that the benchmark is not measuring an impossible

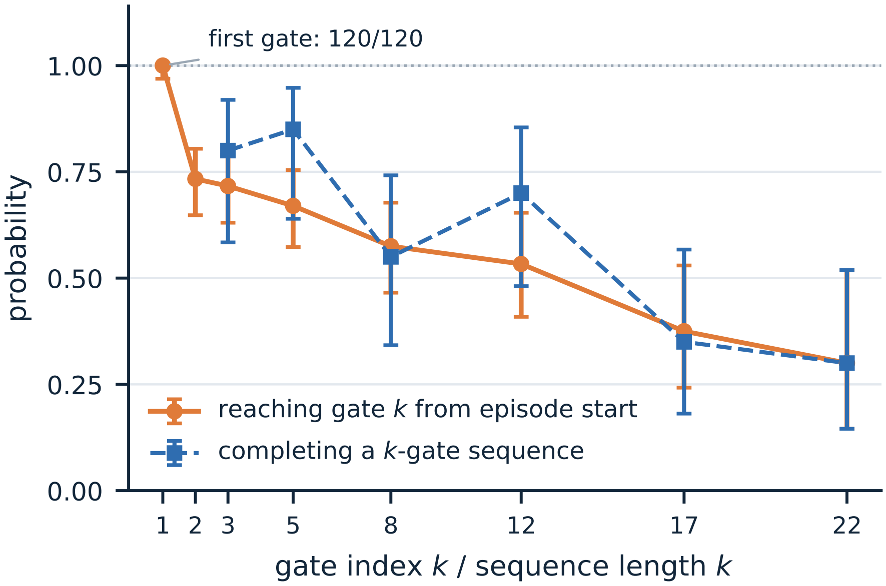

Figure 13: Multi-gate survival of the frozen learned policy on the pre-registered validation set, with Wilson 95% intervals. Solid: the unconditional probability of reaching gate k measured from episode start. Dashed: the probability of completing an independent k-gate sequence. Crossing the first gate succeeds in 120/120 episodes, but the full first transition succeeds in only 88/120, so more than a quarter of episodes are lost at the first re-acquisition. Survivor-conditioned transition rates, which average 0.820 over all 500 cases, conceal exactly this.

task. But it is not *always* right, and the residual 0.75% of false advancements is exactly the error that a benchmark handing progress to the controller would define out of existence.

It buys interpretable failure. Because the referee terminates on a missed gate using privileged geometry, a policy that loses the sequence is scored as having lost the sequence, not as having achieved a low gate count for unclear reasons. This is what allowed us to determine that 90.6% of learned-policy failures on official circuits are sequential rather than physical, which is a substantially more actionable diagnosis than an aggregate completion rate.

11.3. What the Controller Comparison Shows

The Continuous Servo versus Center-then-Commit comparison is a single-variable ablation, and its outcome is condition-dependent rather than uniform: the staged passage costs 1.7% of time on a short planar circuit, loses on a vertically serpentine one, and is decisive on a long mixed one (5/5 against 2/5) and under a moderate current (3/5 against 2/5, with roughly half the collisions).

We read this as a statement about where visual feedback should be trusted rather than about which controller is better. Both controllers use the same detector and the same association logic; they differ only in whether they continue to act on the image during the passage itself. The near-field regime is where the detector is least reliable — bars clip at the image border and the contour becomes partial — and it is also where the aperture margin is smallest. Suspending visual correction there substitutes a stable but open-loop traversal for a closed-loop one driven by a degraded measurement. That trade is favourable exactly when the number of passages is large or the vehicle is being pushed off line, which matches the pattern in Tables 11 and 12. Neither controller is optimal, and we do not present them as such; their value is that a benchmark which ranks them identically would be insensitive to a real design decision.

11.4. What Degradation Under Currents Means

The current results carry a methodological warning that generalizes beyond this benchmark. Both reference controllers achieve a perfect score with zero events on clean Horseshoe Bay and fall to 2/5 and 3/5 with 43.4 and 25.2

Table 16: Pre-registered readiness gate and paired final-holdout outcome for the frozen Generation-1 PPO policy. The thresholds were registered, and the policy frozen with its SHA-256, before either evaluation was run; no manual override was requested or applied.

Criterion	Observed	Bound	Verdict
Readiness gate, 500 validation transition cases and 20 sequences per length			
Universal transition success	0.820	$\geq$ 0.900	**FAIL**
First gate crossing	1.000	$\geq$ 0.970	pass
Target switch	0.882	$\geq$ 0.850	pass
New-target alignment	0.852	$\geq$ 0.850	pass
Completion, 3 gates	0.800	$\geq$ 0.950	**FAIL**
Completion, 5 gates	0.850	$\geq$ 0.900	**FAIL**
Completion, 8 gates	0.550	$\geq$ 0.750	**FAIL**
Completion, 12 gates	0.700	$\geq$ 0.500	pass
Completion, 17 gates	0.350	$\geq$ 0.250	pass
Completion, 22 gates	0.300	$\geq$ 0.250	pass
Out-of-bounds rate	0.000	$\leq$ 0.010	pass
Collision-episode rate	0.066	$\leq$ 0.200	pass
Overall readiness: FAIL (4 of 19 registered criteria missed)			
Final holdout, 30 paired trials over the three official circuits, 10 seeds each			
PPO completion	0/30	—	87 of 510 gates crossed
Rule completion	30/30	—	510 of 510 gates crossed
PPO terminations	30	—	every episode a missed-gate DNF

The readiness gate evaluates 19 criteria in total; the seven protocol and sampling criteria (validation-split match, difficulty match, ladder coverage, case counts) all passed and are omitted here for space. Every PPO episode of the final holdout crossed at least gate 1 and then ended when the referee recorded a missed gate at the next attempted gate; the referee evidence does not support relabelling this as an acquisition or target-switch failure. Holdout access was recorded in a hash-chained ledger of 60 entries.

collisions on the same circuit under a moderate current. Clean-track performance therefore carries almost no information about disturbance robustness, and an evaluation protocol reporting only clean completion would rank these controllers on a dimension that vanishes under the mildest realistic perturbation.

The non-monotonic collision count is a second warning of the same kind. Under the strong profile both controllers record essentially no collisions, not because they handle the disturbance but because they stall after three of twelve gates and never reach the contact-dense part of the course. Any safety metric reported without a progress metric is invertible in this way, which is why Table 12 reports gates, times and events together and why the referee's ranking key (24) orders unfinished participants by progress before collisions.

11.5. Why Multi-Vehicle Evaluation Belongs in the Same Framework

The three-vehicle result isolates a failure that does not exist at the level of an individual controller. Each vehicle in the uncoordinated convoy is running a controller that completes the circuit reliably on its own; the 127.3 mean gate and world collisions arise because two correct controllers pursuing the same aperture with the same centring objective converge to the same point in space. Coordination, not individual competence, is the binding constraint, and it is observable only in a framework where several vehicles share a world and a single team score.

The design choice that makes this measurable is the conjunction in (30): the team finishes only if every vehicle finishes, so a fast leader cannot compensate for a stranded follower. The complementary choice is that a yielding vehicle is still subject to the independent stuck penalty — LF(2) trips it once per run at simultaneous release — so a policy that buys safety by waiting pays for the waiting in the same currency it saves. Together these make the coordination trade-off visible in the score rather than in a side channel.

11.6. Deterministic Baselines and Learned Controllers

The most instructive result of this study is that learned and rule-based controllers fail differently, and that the difference is structural rather than a matter of degree.

The rule controller succeeds by construction on the composition problem: its `LocalCourseTracker` maintains an explicit ordered state and refuses to advance without agreement between camera, acoustic and Doppler evidence (Section 6.3), so chaining is guaranteed by the architecture and the residual risk lies in individual passages. Its cost is speed and generality — it is slower than every PPO checkpoint on the geometries they both solve, by 6.8 to 7.1 s per episode, and its thresholds were designed for this course family.

The learned policies invert the profile. They acquire and cross an individual gate very well, and do it faster, but they hold the ordered sequence in an implicit recurrent-free representation that receives no gate index and no mission length by design (Section 9.2). What they lack is precisely what the rule controller has for free: a committed, verified notion of *which gate is next*. The failure signature follows directly — crossing probability 1.000 at the first gate, 0.733 through the first full transition, and a 90.6% share of missed-gate terminations among official-circuit failures.

This suggests that the productive question is not whether learned control can replace a rule baseline in underwater racing, but where the boundary between learned and structured components should be placed. The hybrid controller in Table 14 is one point on that spectrum — a deterministic backbone with a learned visual servo, reaching 86.7% on the official circuits — and its position is consistent with the diagnosis, since it retains explicit sequence management while learning the part that is genuinely a control problem. We report it separately and make no learned-control claim from it, but its existence supports the reading that the gap is in sequence management rather than in gate-following skill.

11.7. The Benchmark as a Testbed

Taken together, the results suggest that Marine Race Arena's value is less as a leaderboard than as an instrument. The three properties that produced the findings above — a fixed observation boundary, an ordered-sequence termination rule and an independent scorer — are the same three that made those findings legible. The current-free demonstration shows the task is solvable; the current sweep shows clean success does not transfer; the coordination study shows team-level failures that individual evaluation cannot see; and the matched learned-controller benchmark shows a specific, addressable capability gap rather than a diffuse performance deficit. Each of these is a statement about what to work on next, and none of them would have survived a protocol that pooled conditions, reported aggregate completion, or let the controller consult the scorer.

12. Limitations

We state the boundaries of this work explicitly, both because they define what the reported numbers may be used for and because several of them are the natural next steps.

L1. No physical validation, and an unquantified simulation-to-reality gap. Every result in this paper was produced in simulation. No experiment was run on a physical BlueROV2 or in a water tank, and we make no claim that any reported completion rate, time or collision count would be reproduced by a real vehicle. The gap is unquantified rather than small: closing it would require a physical gate course, a calibrated acoustic positioning setup and a characterization of the optical conditions, none of which is attempted here.

L2. Backend fidelity bounds the physical claims. The benchmark inherits the hydrodynamics, rendering and sensor models of its backend. HoloOcean provides a usable and widely adopted approximation, but it is not a validated computational-fluid-dynamics model of a BlueROV2, and the current field of (9) is an analytic superposition of primitives rather than a simulated flow. Conclusions about *relative* controller behaviour are therefore better supported than conclusions about absolute achievable speeds or about the specific current magnitude at which a controller fails.

L3. Optical perception assumptions. The detector of Section 5.1 assumes gates that project to closed, near-rectangular contours of bounded aspect ratio against a background whose edge structure is less rectangular. It was developed for, and validated on, the rendering conditions of these circuits. Turbidity, marine snow, backscatter from an active light source, biofouling on the gate frames, strong caustics and low-light operation are not modelled, and the detector's geometric filters would require re-derivation under any of them.

L4. Gate appearance assumptions. All gates in this study are square frames of identical 1.5 m × 1.5 m inner aperture, and the detector's size, aspect and near-field area gates are calibrated to that geometry. The known-aperture assumption is also what makes the pose extension's known-size distance estimate possible. Heterogeneous gate sizes, non-square apertures, partially occluded gates or gates that must be distinguished by appearance rather than by acoustic association are outside the current scope.

L5. Acoustic sensing is an approximation, not a channel model. The beacon model of (6) applies independent Gaussian perturbations to range, bearing and elevation with Bernoulli dropout and a range cut-off. It does not model multipath, ray bending, Doppler on the acoustic carrier, frequency-dependent absorption or the correlated errors these produce. The inter-vehicle channel of (27) is likewise an acoustic-inspired transport with range-dependent latency and independent packet loss; its loss probability, range, latency and freshness window were not independently swept, and no claim about acoustic communication performance is made.

L6. The current sweep is narrow. Quantitative current results are restricted to two named profiles on one circuit (Horseshoe Bay). The `strong` profile turned out to be a saturation condition rather than a graded difficulty, so the informative disturbance evidence rests effectively on a single profile and five seeds. A finer sweep, and the same sweep on the other two circuits, would be needed to characterize the degradation curve rather than two points on it.

L7. Two baseline controller families. The rule-based evidence covers exactly two controllers that differ in one design decision. This is deliberate — it makes the comparison a clean ablation — but it means the benchmark has not yet been exercised against structurally different autonomy strategies, such as a model-predictive controller, a cascaded design with an inner DVL velocity loop, or a planner that reasons about several gates ahead. We therefore cannot claim that the difficulty ordering across the three circuits is a property of the circuits rather than of these two controllers.

L8. Limited fleet scale and diversity. Fleet evidence covers two homogeneous vehicles at a 90 s gap and three heterogeneous vehicles at two gaps, on one clean circuit, with three or five seeds. Larger fleets, fleets under current or with obstacles, formation-keeping objectives, cooperative overtaking, dynamic role assignment and competitive multi-team racing are all unaddressed. The coordination result in particular rests on three seeds per condition.

L9. The pose-aware perception extension is not validated. As reported in Section 5.3, the exploratory pose front end recovers a pose in only 11.7% of frames on a dedicated oblique-view rig, with a median plane-yaw error of 48.2° and orientation-class and sign accuracies of 0.00. It is accurate in the near-frontal regime the controller actually operates in (Table 6) and unreliable outside it. It is implemented and released, but it supports no claim in this paper and should not be used as a pose estimator.

L10. The learned-control results are a snapshot of an open problem. The learned policies reported here are feed-forward PPO and behaviour-cloning checkpoints trained on procedural geometry. The frozen policy failed a pre-registered readiness gate on four of nineteen criteria and completed 0 of 30 official-circuit holdout trials, and the best checkpoint of the matched benchmark reaches 40% on those circuits. We report a diagnosis — the deficit is in ordered-sequence management rather than in gate-following — and not a solution. Architectures with explicit memory, imitation-based bootstrapping and interactive dataset aggregation are natural responses to that diagnosis and are not evaluated here. A separate line of work along those lines was in progress while this manuscript was written and is deliberately excluded: no result from it appears anywhere in this paper.

L11. The official circuits are no longer unseen. An exploratory holdout evaluation opened Horseshoe Bay, Vertical Serpent and Mixed Endurance, and its access ledger records all sixty accesses. They must therefore not be treated as held-out circuits for any policy trained after that point, and every learned-controller result on them in this paper is labelled a retrospective diagnostic benchmark. Future generalization claims require a sealed circuit set that has not been opened.

L12. Three circuits do not span underwater autonomy. The official track set was designed to vary length, verticality and sequencing, and it does so — 12 to 22 gates, 93.8 to 206.3 m, planar to serpentine. It nonetheless covers a narrow slice of the underwater autonomy problem. Docking, manipulation, inspection, long-horizon navigation with drift, adversarial or dynamic environments and mission-level planning are all outside the benchmark's task definition, and results here should not be read as evidence about them.

13. Reproducibility

Every number in this paper is traceable to a released artifact through a recorded chain of identifiers. This section documents that chain so a reader can verify a value, re-derive a table, or run an equivalent experiment.

13.1. Repository and Environment

The implementation, the track configurations, the controllers, the evaluation harness and the aggregated result artifacts are available in a public repository [21]. The benchmark environment is HoloOcean 2.3.0 with its Ocean world, Python 3.9.25 and the pinned dependency set in `requirements.txt`; the learning extension adds a separate pinned set in `requirements-rl.txt` (Gymnasium 1.0.0, PyTorch 2.8.0, Stable-Baselines3 2.7.1) that is deliberately installed into a distinct environment so the benchmark environment used for the frozen matrix remains unchanged. Runs reported here were executed on Windows 10 build 26200.

13.2. Configuration-Driven Experiments

A race is fully specified by one JSON file (Section 4): world bounds and environment, the ordered gate sequence with centres, passage directions and aperture sizes, the beacon network and its noise parameters, the current profiles, the obstacle generation rules, the participants with their vehicles, sensors, controllers and release delays, and the referee's validation margins, penalties and scoring rules. A single entry point runs any of them, and a controller is loaded by alias, by dotted module path or by file path plus class name without modifying the package (Section 6). Reproducing a reported condition therefore requires the track file, the controller identifier and the seed, all of which are recorded per run.

13.3. Seeds and Determinism

Beacon packet randomness is seeded by the run seed, the transmitter, the receiver and the transmission index, so reception is independent of participant count and of the order in which controllers are polled; a fleet run is reproducible for this reason. Inter-vehicle packet loss and procedural course generation draw from seeded generators. For the learning extension, seed bands are registered by role — training, validation, test and final holdout — in a seed registry committed to the repository, and each evaluation records both requested and completed seeds. The matched benchmark of Section 10.6 evaluated every controller on the same seed set and the same generated geometries, which is what makes its paired statistics valid.

13.4. Manifests, Fingerprints and Hashes

Three levels of provenance are recorded.

Per-run. Each run emits a line-delimited event log and a machine-readable summary (R5), plus metadata giving the track, controller, seed, command line, requested and actual adapter, fallback-disabled status, control timestep, requested and measured current vector, and the observation and action contract versions.

Per-experiment. The 78-run matrix carries a complete experiment manifest recording the experiment-generation commit (`df098ef4...`), a source-tree fingerprint over the package's Python and JSON files (`e7d31077...`), the HoloOcean version, the Python version, the operating system, the fallback status and the physical current-coupling status for every run. It also asserts matrix completeness: 78 of 78 expected runs, no missing and no unexpected metadata.

Per-artifact. Learned-controller evaluations additionally record the model path and its SHA-256, the track file SHA-256 and the suite commit. The frozen Generation-1 policy carries a freeze record written with an exclusive create, containing the checkpoint SHA-256 (`60ffdca1...`), the git commit at freeze time and the transition count, and the readiness thresholds are hashed inside that record so that a later edit to the defaults would be detectable. Holdout accesses are recorded in a hash-chained ledger of 60 entries.

13.5. Regenerating Tables and Figures

All result tables and figures in this paper are *post-processing* of existing artifacts: no command listed below launches the simulator or modifies a raw artifact.

```
# Result tables from the frozen 78-run matrix (also runs the penalty-identity check)
python article/regenerate_tables.py --check

# Track layouts and the controller-comparison plot, from track JSON and artifacts
python article/figures/generate_figures.py

# Journal figures: perception panels and learning plots, from committed artifacts
python article_journal/scripts/make_perception_figure.py
python article_journal/scripts/make_learning_figures.py

# Manuscript
cd article_journal && latexmk -pdf -interaction=nonstopmode -halt-on-error main.tex
```

Running `regenerate_tables.py` against the frozen matrix reproduces the clean-track, current, fleet and coordination values of Tables 11, 12 and 13 exactly, and also verifies the penalty identity $T^{\mathrm{pen}} = T^{\mathrm{official}} + P$ on every finished run. The two journal figure scripts write a provenance JSON alongside each output recording their input files and asserting that no simulator was launched.

13.6. Re-Running Experiments

Reproducing a run rather than a table requires HoloOcean and the recorded seed. Every experimental command is stored verbatim in the per-run metadata; a clean single-vehicle run has the form

```
python -m marine_race_arena.scripts.run_marine_race \
  --track marine_race_arena/tracks/marine_race_horseshoe_bay.json \
  --benchmark-task clean_gate \
  --controller rule_gate_center_then_commit \
  --adapter holoocean --seed 0 --dt 0.033 \
  --duration 560.0 --current-profile none --official
```

and the current-free demonstration of Section 10.2 is driven by committed scripts that pass `-current-profile none` and assert `currents_actual` = $[0, 0, 0]$ in the manifest before any episode runs. Two conventions are worth restating for anyone attempting a comparison. First, the engine-free fallback adapter exists for deterministic unit testing and must be disabled for any reported result; the manifest field `adapter_actual` records which adapter actually ran. Second, the control timestep must match the condition being reproduced: $\Delta t = 0.033$ s for the 78-run matrix and $\Delta t = 0.1$ s for the current-free demonstration and the matched learned-controller benchmark.

13.7. Updating the Learning Results

The learning section is deliberately structured so that its tables can be refreshed without rewriting the manuscript. A new frozen checkpoint is added by recording its SHA-256 in the benchmark package manifest, running the same matched suite on the same registered seeds, and regenerating Tables 14–15 and Fig. 12 from the resulting aggregate files. The methodology of Section 9 — observation encoding, reward, curriculum, readiness gate and evaluation protocol — is independent of any particular checkpoint and does not change when the numbers do.

14. Conclusion and Future Work

This paper presented Marine Race Arena, a reproducible benchmark and evaluation framework for onboard-only autonomous underwater gate racing. Its organizing principle is a strict, framework-enforced separation between autonomy and evaluation: a controller reads only participant-local time, allow-listed onboard sensing, physically received acoustic packets and optional teammate messages, while an independent referee uses privileged simulator state to validate ordered gate crossings, charge penalties, terminate runs and compute scores that never return to the control loop. On that contract we built an onboard gate-perception and target-association front end, two interpretable rule-based reference controllers, a cooperative fleet layer with a distributed leader–follower policy over an acoustic-inspired channel, and a learning-integration path that exposes the same observation and action interface to reinforcement-learning agents.

The experiments support four conclusions. The task is achievable under the full contract: the reference controller completed all three official circuits in 9 of 9 current-free runs, with no collision, out-of-bounds or wrong-direction event, and with a manifest that verifies the real adapter and zero currents. The benchmark discriminates: two controllers that differ in a single design decision — whether to keep servoing on the image during a passage — score identically on a short planar circuit and diverge to 5/5 against 2/5 on a long mixed one. Clean-track success does not transfer: the same controllers fall to 3/5 and 2/5 with tens of collisions on the same circuit under a moderate current, and the non-monotonic collision count under the strong profile shows that a safety metric reported without a progress metric can invert a ranking. And learned controllers integrate cleanly but fail in a specific, diagnosable way: across 474 matched episodes they complete 90.6–100% of short generated geometries and are significantly faster there than the rule baseline, yet reach only 20–40% on full circuits, with 90.6% of those failures being missed-gate terminations rather than collisions.

The recurring finding is that the difficulty of underwater gate racing is concentrated in *ordered-sequence management* rather than in individual gate following. The unconditional survival curve makes this precise: the frozen learned policy crosses its first gate with probability 1.000 and completes the full first transition with probability 0.733, so more than a quarter of episodes are lost at the very first re-acquisition, a fact that survivor-conditioned rates conceal. Making that distinction visible required all three of the benchmark's defining properties — a fixed observation boundary, an ordered-sequence termination rule and an impartial scorer — and it is the clearest evidence we can offer that the evaluation layer earns its place above the simulator.

The work has clear boundaries, stated in Section 12: no physical validation, a backend whose fidelity bounds the absolute claims, a narrow current sweep on one circuit, two baseline controller families, a small fleet, an unvalidated pose-aware perception extension, and learned-control results that diagnose an open problem rather than solve it.

Four directions follow directly. *(i)* Sequence-aware learned control: the measured deficit is chaining, not gate following, which points to policies with explicit memory, to imitation-based bootstrapping from the deterministic controller, and to interactive dataset aggregation that supplies corrections at the transition points where compounding error originates. *(ii)* Disturbance rejection as a benchmark axis: a finer current sweep across all three circuits, together with controllers that explicitly reject the disturbance — for instance a cascaded design servoing DVL-measured body velocity beneath outer beacon and vision guidance, with no access to current telemetry. *(iii)* Richer multi-robot evaluation: larger fleets, coordination under current and obstacles, formation keeping, cooperative overtaking, and competitive multi-team racing with team-isolated scoring under the same referee. *(iv)* Physical validation: transferring a gate course and the observation contract to a real BlueROV2 to quantify the simulation-to-reality gap that this work leaves open. For all four, the contribution offered here is not a controller but the instrument that makes the comparison meaningful.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Data availability

The benchmark implementation, the track configurations, the evaluation harness and the aggregated result artifacts are publicly available at `https://github.com/AndreaBedei1/HoloDroneCompetition`. Section 13 describes the artifact layout and the regeneration commands.

References

[1] J. Betz, H. Zheng, A. Liniger, U. Rosolia, P. Karle, M. Behl, V. Krovi, R. Mangharam, Autonomous vehicles on the edge: A survey on autonomous vehicle racing, IEEE Open Journal of Intelligent Transportation Systems 3 (2022) 458–488. doi:10.1109/OJITS.2022.3181510.

[2] M. O'Kelly, H. Zheng, D. Karthik, R. Mangharam, F1TENTH: An open-source evaluation environment for continuous control and reinforcement learning, in: Proceedings of the NeurIPS 2019 Competition and Demonstration Track, Vol. 123 of Proceedings of Machine Learning Research, 2020, pp. 77–89.
URL `https://proceedings.mlr.press/v123/o-kelly20a.html`

[3] S. Hoffmann, S. Sagmeister, T. Betz, J. Bongard, S. Büttner, D. Ebner, D. Esser, G. Jank, S. Goblirsch, A. Langmann, M. Leitenstern, L. Ögretmen, P. Pitschi, A.-K. Schwehn, C. Schröder, M. Weinmann, F. Werner, B. Lohmann, J. Betz, M. Lienkamp, Head-to-head autonomous racing at the limits of handling in the A2RL challenge, arXiv preprint arXiv:2602.08571Abu Dhabi Autonomous Racing League; preprint, not peer reviewed (2026). arXiv:2602.08571.
URL `https://arxiv.org/abs/2602.08571`

[4] P. Foehn, D. Brescianini, E. Kaufmann, T. Cieslewski, M. Gehrig, M. Muglikar, D. Scaramuzza, AlphaPilot: Autonomous drone racing, in: Robotics: Science and Systems (RSS), 2020. doi:10.15607/RSS.2020.XVI.081.
URL `https://www.roboticsproceedings.org/rss16/p081.html`

[5] E. Kaufmann, L. Bauersfeld, A. Loquercio, M. Müller, V. Koltun, D. Scaramuzza, Champion-level drone racing using deep reinforcement learning, Nature 620 (7976) (2023) 982–987. doi:10.1038/s41586-023-06419-4.

[6] M. M. M. Manhães, S. A. Scherer, M. Voss, L. R. Douat, T. Rauschenbach, UUV simulator: A Gazebo-based package for underwater intervention and multi-robot simulation, in: OCEANS 2016 MTS/IEEE Monterey, 2016, pp. 1–8. doi:10.1109/OCEANS.2016.7761080.

[7] P. Cieślak, Stonefish: An advanced open-source simulation tool designed for marine robotics, with a ROS interface, in: OCEANS 2019 - Marseille, 2019, pp. 1–6. doi:10.1109/OCEANSE.2019.8867434.

[8] M. M. Zhang, W.-S. Choi, J. Herman, D. Davis, C. Vogt, M. McCarrin, Y. Vijay, D. Dutia, W. Lew, S. Peters, B. Bingham, DAVE aquatic virtual environment: Toward a general underwater robotics simulator, in: Proceedings of the IEEE/OES Autonomous Underwater Vehicles Symposium (AUV), 2022, pp. 1–8. doi:10.1109/AUV53081.2022.9965808.
URL `https://arxiv.org/abs/2209.02862`

[9] E. Potokar, S. Ashford, M. Kaess, J. G. Mangelson, HoloOcean: An underwater robotics simulator, in: Proceedings of the IEEE International Conference on Robotics and Automation (ICRA), 2022, pp. 3040–3046. doi:10.1109/ICRA46639.2022.9812353.
URL `https://www.ri.cmu.edu/publications/holoocean-an-underwater-robotics-simulator/`

[10] S. Chu, Z. Huang, Y. Li, M. Lin, D. Li, I. Carlucho, Y. R. Petillot, C. Yang, MarineGym: A high-performance reinforcement learning platform for underwater robotics, in: 2025 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS), 2025, pp. 17146–17153. doi:10.1109/IROS60139.2025.11246146.
URL `https://ieeexplore.ieee.org/document/11246146`

[11] M. von Benzon, F. F. Sørensen, E. Uth, J. Jouffroy, J. Liniger, S. Pedersen, An open-source benchmark simulator: Control of a BlueROV2 underwater robot, Journal of Marine Science and Engineering 10 (12) (2022) 1898. doi:10.3390/jmse10121898.

[12] A. Dosovitskiy, G. Ros, F. Codevilla, A. Lopez, V. Koltun, CARLA: An open urban driving simulator, in: Proceedings of the Conference on Robot Learning (CoRL), Vol. 78 of Proceedings of Machine Learning Research, 2017, pp. 1–16.
URL `https://proceedings.mlr.press/v78/dosovitskiy17a.html`

[13] Autonomous Robotics Research Center, Technology Innovation Institute, A2RL: Abu dhabi autonomous racing league, https://a2rl.io, autonomous car and drone racing championship; accessed 2026-06-27 (2024).

[14] G. Brockman, V. Cheung, L. Pettersson, J. Schneider, J. Schulman, J. Tang, W. Zaremba, OpenAI gym, arXiv preprint arXiv:1606.01540 (2016). arXiv:1606.01540.
URL https://arxiv.org/abs/1606.01540

[15] V. Makoviychuk, L. Wawrzyniak, Y. Guo, M. Lu, K. Storey, M. Macklin, D. Hoeller, N. Rudin, A. Allshire, A. Handa, G. State, Isaac gym: High performance GPU-based physics simulation for robot learning, in: Proceedings of the NeurIPS 2021 Track on Datasets and Benchmarks, 2021. arXiv:2108.10470.
URL https://arxiv.org/abs/2108.10470

[16] M. Stojanovic, J. Preisig, Underwater acoustic communication channels: Propagation models and statistical characterization, IEEE Communications Magazine 47 (1) (2009) 84–89. doi:10.1109/MCOM.2009.4752682.

[17] Y. Yang, Y. Xiao, T. Li, A survey of autonomous underwater vehicle formation: Performance, formation control, and communication capability, IEEE Communications Surveys & Tutorials 23 (2) (2021) 815–841. doi:10.1109/COMST.2021.3059998.

[18] T. Yan, Z. Xu, S. A. Gadsden, Formation control of multiple autonomous underwater vehicles: A review, Intelligence & Robotics 3 (1) (2023) 1–22. doi:10.20517/ir.2023.01.

[19] J. Schulman, F. Wolski, P. Dhariwal, A. Radford, O. Klimov, Proximal policy optimization algorithms, arXiv preprint arXiv:1707.06347 (2017). arXiv:1707.06347.
URL https://arxiv.org/abs/1707.06347

[20] A. Raffin, A. Hill, A. Gleave, A. Kanervisto, M. Ernestus, N. Dormann, Stable-baselines3: Reliable reinforcement learning implementations, Journal of Machine Learning Research 22 (268) (2021) 1–8.
URL https://jmlr.org/papers/v22/20-1364.html

[21] A. Bedei, HoloDroneCompetition: Marine Race Arena repository, https://github.com/AndreaBedei1/HoloDroneCompetition (2026).